

```

873     error = f->f_op->open(inode, f);
874     if (error)
875         goto cleanup_all;
876     intent_release(it);
877 }
...
891 return f;
...
907 }
-----

```

第 852 行：调用 `get_empty_filp()` 来分配文件结构体。

第 855~856 行：用传入 `open` 系统调用的标志设置文件结构体的 `f_flags` 字段。把传给 `open` 系统调用的访问模式先转换成路径名查找函数所需的格式，然后用它来设置文件结构体的 `f_mode` 字段。

第 866~869 行：把文件结构体的 `f_dentry` 字段设置成指向与文件路径名有关的目录项结构体。把 `f_vfsmnt` 字段设置成指向具体文件系统的 `vmfsmount` 结构体。`f_pos` 被设置为 0，表示 `file_offset` 的起始位置位于文件的开头。`f_op` 字段被设置成指向由文件索引节点所指向的操作表。

第 870 行：调用 `file_move()` 例程把已打开的文件的文件结构体插入其所在文件系统的超级块链表中。

第 872~877 行：此处，调用 `open` 函数的下层函数。如果文件中还有多个特定于文件的功能要执行，以打开文件，则调用这个打开函数。如果文件的操作表中含有打开例程，也调用这样的打开例程。

至此，已经涵盖了 `dentry_open_it()` 例程的所有内容。

到 `filp_open()` 结束时，将刚分配的文件结构体插入超级块 `s_files` 字段的头部，同时让 `f_dentry` 指向 `dentry` 对象、`f_vfsmount` 指向 `vfsmount` 对象、`f_op` 指向索引节点的文件操作表 `i_fop`，并把 `f_flags` 设置成访问标志，把 `f_mode` 设置成传给 `open()` 调用的权限模式。

第 944 行：`fd_install()` 例程设置 `fd` 数组指针，使之指向由 `filp_open()` 返回的文件对象的地址。换句话说，就是设置 `current->files->fd[fd]`。

第 947 行：`putname()` 例程释放文件名所占用的内核空间。

第 949 行：返回文件描述符 `fd`。

第 952 行：`put_unused_fd()` 例程清除已经分配的描述符。当文件对象创建失败时调用该例程。

总之，`open()` 系统调用的多级调用过程看起来是下面这样的。

`sys_open`:

- `getname()`：把文件名传送到内核空间；
- `get_unused_fd()`：获得下一个可用的文件描述符；
- `filp_open()`：创建 `nameidata` 结构体；

- `open_namei()`: 初始化 `nameidata` 结构体;
- `dentry_open()`: 创建并初始化文件对象;
- `fd_install()`: 将 `current->files->fd[fd]` 设置成文件对象;
- `putname()`: 回收为文件名分配的内核空间。

图 6-16 说明了被初始化和设置的结构, 并标识了完成这些初始化和设置的例程。

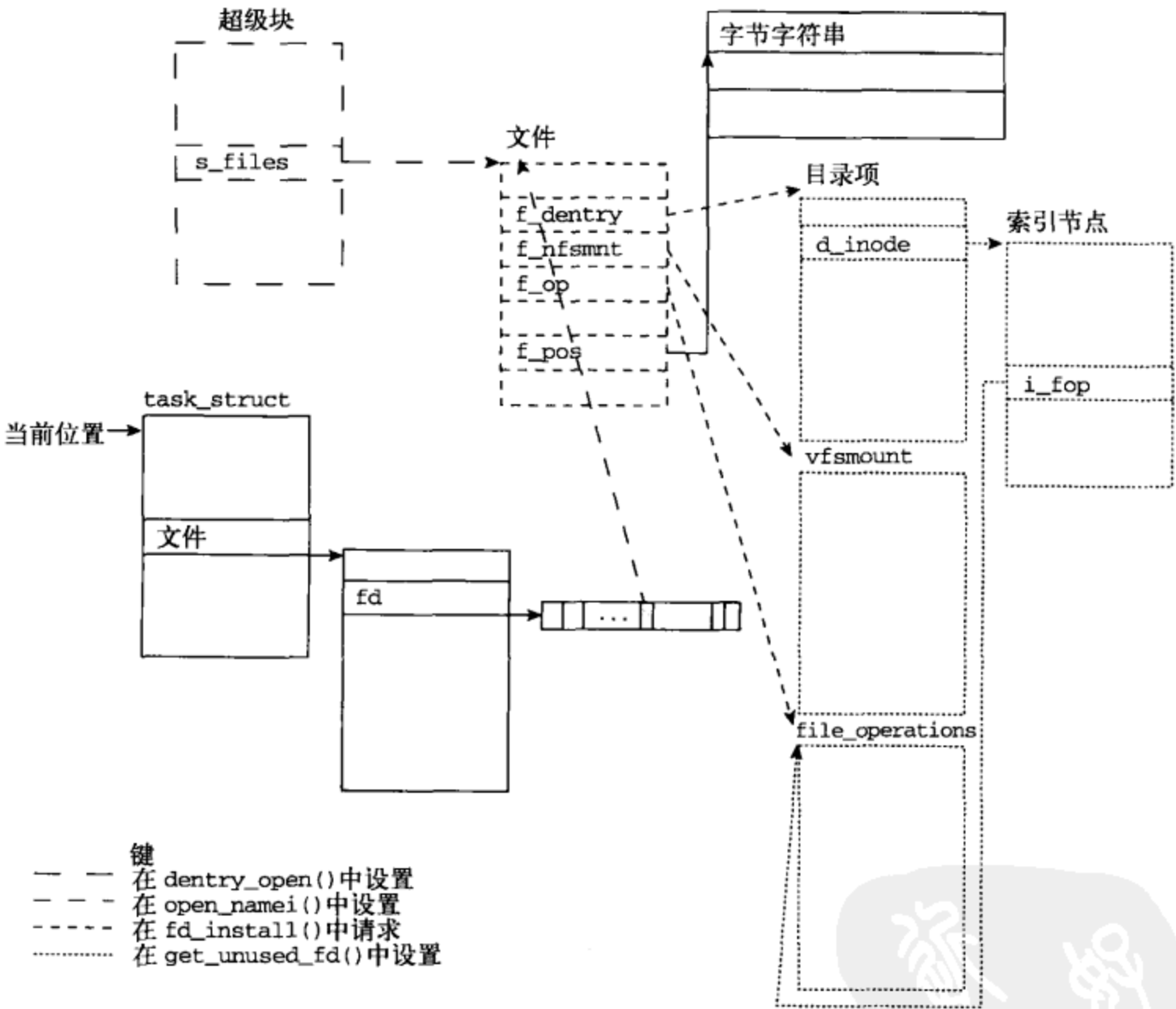


图 6-16 文件系统的结构

表 6-10 给出了 `sys_open()` 返回的一些错误以及找到这些错误的内核例程。

表6-10 `sys_open()` 返回的错误

错 误 码	说 明	返回错误的函数
ENAMETOOLONG	路径名太长	<code>getname()</code>
ENOENT	文件不存在 (标志 <code>O_CREAT</code> 也没有设置)	<code>getname()</code>
EMFILE	进程打开的文件数达到最大值	<code>get_unused_fd()</code>
ENFILE	系统中打开的文件数达到最大值	<code>get_unused_filp()</code>

6.5.2 close()

进程使用完文件后，就发出 close() 系统调用：

```
-----
synopsis
#include <unistd.h>

int close(int fd);
-----
```

close 系统调用把将要关闭的文件的文件描述符作为参数。在标准 C 程序中，在程序终止时隐含着对 close() 的调用。让我们深入分析一下 sys_close() 的代码：

```
-----
fs/open.c
1020 asmlinkage long sys_close(unsigned int fd)
1021 {
1022     struct file * filp;
1023     struct files_struct *files = current->files;
1024
1025     spin_lock(&files->file_lock);
1026     if (fd >= files->max_fds)
1027         goto out_unlock;
1028     filp = files->fd[fd];
1029     if (!filp)
1030         goto out_unlock;
1031     files->fd[fd] = NULL;
1032     FD_CLR(fd, files->close_on_exec);
1033     __put_unused_fd(files, fd);
1034     spin_unlock(&files->file_lock);
1035     return filp_close(filp, files);
1036
1037 out_unlock:
1038     spin_unlock(&files->file_lock);
1039     return -EBADF;
1040 }
-----
```

第 1023 行：当前 task_struct 的 files 字段指向与我们文件对应的 files_struct。

第 1025~1030 行：这几行首先给文件加锁，以免遇到同步问题。然后，检查文件描述符是否有效。如果文件描述符编号大于这个文件所允许的最大值，就删除锁并返回错误码-EBADF。否则将获得文件结构体的地址。如果文件描述符索引不能产生一个文件结构体，我们也删除锁并返回错误，因为没有文件要关闭。

第 1031~1032 行：此处，将 current->files->fd[fd] 设置为 NULL，即删除指向文件对象的指针。同时也在 files->close_on_exec 所指向的文件描述符集中清除该文件描述符的相应位。因为文件描述符被关闭，所以进程不必担心在调用 exec() 时还会涉及它。

第 1033 行：因为这个文件不再是打开的，内核例程 __put_unused_fd() 在文件描述符集 files->open_fds 中清除这个文件描述符的相应位。__put_unused_fd() 采取了一些措施，以确保遵循文件描述符的“最小可用索引”的分配原则：

```

-----
fs/open.c
897 static inline void __put_unused_fd(struct files_struct *files, unsigned int fd)
898 {
899     __FD_CLR(fd, files->open_fds);
900     if (fd < files->next_fd)
901         files->next_fd = fd;
902 }
-----

```

第 890~891 行：next_fd 字段保存下一个将要分配的文件描述符的值。如果当前文件描述符的值小于 files->next_fd 中的值，则把这个域设置为当前文件描述符的值。这样做可以保证文件描述符总是遵循“最小可用值”的原则进行分配。

第 1034~1035 行：现在释放文件锁，并且把控制权传递给 filp_close() 函数，该函数负责为 close 系统调用返回合适的值。filp_close() 函数完成 close 系统调用的大部分工作。接下来详细分析一下 filp_close() 例程：

```

-----
fs/open.c
987 int filp_close(struct file *filp, fl_owner_t id)
988 {
989     int retval;
990     /* Report and clear outstanding errors */
991     retval = filp->f_error;
992     if (retval)
993         filp->f_error = 0;
994
995     if (!file_count(filp)) {
996         printk(KERN_ERR "VFS: Close: file count is 0\n");
997         return retval;
998     }
999
1000     if (filp->f_op && filp->f_op->flush) {
1001         int err = filp->f_op->flush(filp);
1002         if (!retval)
1003             retval = err;
1004     }
1005
1006     dnotify_flush(filp, id);
1007     locks_remove_posix(filp, id);
1008     fput(filp);
1009     return retval;
1010 }
-----

```

第 991~993 行：这几行清除任何明显的错误。

第 995~997 行：这是关闭文件时进行完整性检查的必要条件。file_count 为 0 的文件应该早已关闭。因此，在这种情况下，filp_clos 将返回一个错误。

第 1000~1001 行：调用文件操作函数 flush()（如果定义了的话）。它的功能取决于具体的文件系统。

第 1008 行：调用 `fput()` 来释放文件结构体。这个例程完成的操作包括：调用文件操作函数 `release()`，删除指向目录项对象和 `vfsmount` 对象的指针，最后，释放文件对象。

`close()` 系统调用的多级调用过程如下。

sys_close():

❑ `_put_unused_fd()`：把文件描述符归还给可用池。

❑ `filp_close()`：准备即将清除的文件对象。

❑ `fput()`：清除文件对象。

表 6-11 给出了 `sys_close()` 返回的一些错误以及找到这些错误的内核例程。

表6-11 `sys_close()` 返回的错误

错 误	函 数	说 明
EBADF	<code>sys_close()</code>	无效的文件描述符

6.5.3 read()

当用户级程序调用函数 `read()` 时，Linux 把它转换成系统调用 `sys_read()`：

```
-----
fs/read_write.c
272 asmlinkage ssize_t sys_read(unsigned int fd, char __user * buf, size_t count)
273 {
274     struct file *file;
275     ssize_t ret = -EBADF;
276     int fput_needed;
277
278     file = fget_light(fd, &fput_needed);
279     if (file) {
280         ret = vfs_read(file, buf, count, &file->f_pos);
281         fput_light(file, fput_needed);
282     }
283
284     return ret;
285 }
-----
```

第 272 行：`sys_read()` 用于获取文件描述符、用户空间的缓冲区指针以及从文件读入缓冲区的字节数。

第 273~282 行：文件的查找是通过 `fget_light()` 把文件描述符转换成文件指针来完成的。然后调用 `vfs_read()` 以完成所有主要的工作。每个 `fget_light` 都必须和 `fput_light()` 成对出现，因此完成 `vfs_read()` 后就调用 `fput_light()`。

系统调用 `sys_read()` 把控制权传给 `vfs_read()`，因此，继续追踪 `vfs_read()` 吧：

```
-----
fs/read_write.c
200 ssize_t vfs_read(struct file *file, char __user *buf, size_t count,
loff_t *pos)
201 {
-----
```

```

202 struct inode *inode = file->f_dentry->d_inode;
203 ssize_t ret;
204
205 if (!(file->f_mode & FMODE_READ))
206     return -EBADF;
207 if (!file->f_op || (!file->f_op->read && \                !file->f_op->aio_read))
208     return -EINVAL;
209
210 ret = locks_verify_area(FLOCK_VERIFY_READ, inode,
211 file, *pos, count);
212 if (!ret) {
213     ret = security_file_permission (file, MAY_READ);
214     if (!ret) {
215         if (file->f_op->read)
216             ret = file->f_op->read(file,
217 buf, count, pos);
218         else
219             ret = do_sync_read(file, buf,
220 count, pos);
221         if (ret > 0)
222             dnotify_parent(file->f_dentry,
223 DN_ACCESS);
224     }
225 }
226 return ret;
227 }

```

第 200 行：前 3 个参数都是经由 `sys_read()` 的参数传入（或者转换而成）的。第四个参数是文件内部的偏移，读操作从该偏移处开始。如果显式地调用 `vfs_read()`，文件偏移可能不为 0，因为 `vfs_read()` 可能是从内核内部被调用的。

第 202 行：保存指向文件索引节点的指针。

第 205~208 行：对文件操作结构进行基本检查，确保已经定义了读操作或异步读操作。如果没有定义读操作，或者找不到操作表，该函数就在此处返回 `EINVAL` 错误。这个错误表示文件描述符所关联的结构体不能进行读操作。

第 210~214 行：检验要读的区域是不是没有上锁，并检验文件是不是已经被授权读。只要这两个条件不同时满足，就通知该文件的父目录（见第 218~219 行）。

第 215~217 行：这是 `vfs_read()` 的主要内容。如果已经定义了读文件操作，就调用它；否则，调用 `do_sync_read()`。

在追踪过程中，我们追踪的是标准的文件操作 `read()` 而不是 `do_sync_read()`。随后，可以清楚地看到，在底层，它们最终殊途同归。

1. 从通用文件系统层到特定文件系统层

从通用文件系统层过渡到特定文件系统层，这是在众多抽象概念中我们遇到的第一个。图 6-17 说明了文件结构体是如何指向特定文件系统的表或操作的。回想一下，当调用 `read_inode()` 时，索引节点的信息被填充，其中就包括让 `fop` 字段指向由特定文件系统（例如 `ext2`）所定义的合适的操作表。

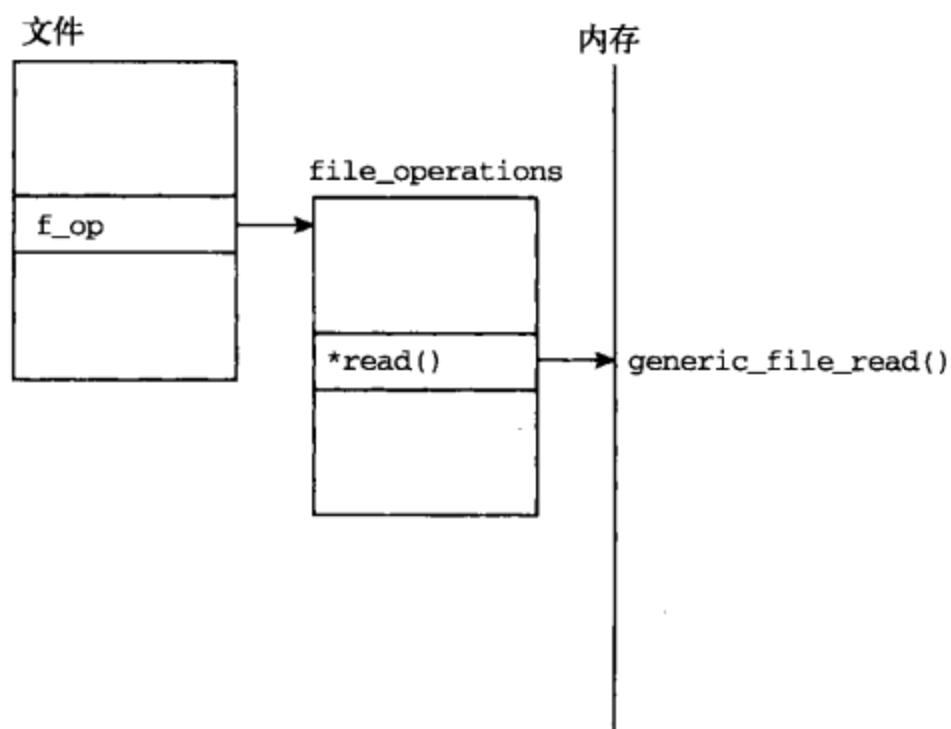


图 6-17 文件操作

当创建或安装文件时，特定文件系统层初始化其文件操作结构。因为我们正在操作的是 ext2 文件系统上的文件，所以，其文件操作结构如下所示：

```

-----
fs/ext2/file.c
42 struct file_operations ext2_file_operations = {
43     .llseek    = generic_file_llseek,
44     .read      = generic_file_read,
45     .write     = generic_file_write,
46     .aio_read  = generic_file_aio_read,
47     .aio_write = generic_file_aio_write,
48     .ioctl     = ext2_ioctl,
49     .mmap      = generic_file_mmap,
50     .open      = generic_file_open,
51     .release   = ext2_release_file,
52     .fsync     = ext2_sync_file,
53     .readv     = generic_file_readv,
54     .writev    = generic_file_writev,
55     .sendfile  = generic_file_sendfile,
56 };
-----
  
```

可以看到，几乎对每一个文件操作，ext2 文件系统都决定采用 Linux 的默认值。那么，文件系统应该在何时实现它自己的文件操作呢？当一个文件系统与 UNIX 的文件系统差异很大时，也许有必要给出几个额外的步骤使得 Linux 可以与这个文件系统进行交互。例如，基于 MSDOS 或 FAT 的文件系统必须实现它们自己的写操作，但是可以使用通用的读操作^①。

了解了 ext2 文件系统层如何把控制权传递给通用文件系统层后，再来考察函数 `generic_file_read()`：

① 详情参见 `fs/fat/file.c`。

```

-----
mm/filemap.c
924 ssize_t
925 generic_file_read(struct file *filp, char __user *buf, size_t count, loff_t *ppos)
926 {
927     struct iovec local_iov = { .iov_base = buf, .iov_len = count };
928     struct kiocb kiocb;
929     ssize_t ret;
930
931     init_sync_kiocb(&kiocb, filp);
932     ret = __generic_file_aio_read(&kiocb, &local_iov, 1, ppos);
933     if (-EIOCBQUEUED == ret)
934         ret = wait_on_sync_kiocb(&kiocb);
935     return ret;
936 }
937
938 EXPORT_SYMBOL(generic_file_read);
-----

```

第 924~925 行：注意，相同的参数沿着上一层的读函数被传递过来。这些参数有：文件指针 `filp`、指向内存缓冲区的指针 `buf`（文件内容将被读到这个缓冲区中）、读入的字符数 `count`、从文件中读数据的起始位置 `ppos`。

第 927 行：创建 `iovec` 结构，该结构包含用户空间缓冲区的地址和长度，读入的数据被存入这个缓冲区中。

第 928~931 行：用文件指针初始化 `kiocb` 结构（`kiocb` 代表内核 I/O 控制块）。

第 932 行：读操作的大部分工作在通用的异步文件读函数中完成。

异步 I/O 操作

`kiocb` 和 `iovec` 是 Linux 内核中协助异步 I/O 操作的两个数据类型。

当进程希望执行输入输出操作，但并不需要等一会儿就马上得到操作结果时，异步 I/O 是非常合适的。对于高级的 I/O 环境尤其适用，因为你可以允许设备对 I/O 请求（而不是进程）进行排序和调度。

在 Linux 中，I/O 向量（`iovec`）表示内存区的地址范围，它的定义如下：

```

-----
include/linux/uio.h
20 struct iovec
21 {
22     void __user *iov_base; /* BSD uses caddr_t (1003.1g requires
    void *) */
23     __kernel_size_t iov_len; /* Must be size_t (1003.1g) */
24 };
-----

```

它只不过是指向内存区的指针和内存区的长度。

内核 I/O 控制块（`kiocb`）是辅助管理 I/O 向量所需的结构，它帮助管理 I/O 向量如何异步地操作以及何时操作。

`__generic_file_aio_read()` 函数使用 `kiocb` 和 `iovec` 结构直接读取 `page_cache`。

第 933~935 行：发出读请求后一直等待，直到读请求完成并返回读操作的结果为止。

回想 `vfs_read()` 中的 `do_sync_read()` 这条线；它最终会通过另一条线调用这一相同的函数。接下来通过考察 `__generic_file_aio_read()` 继续对文件 I/O 进行追踪：

```
-----
mm/filemap.c
835 ssize_t
836 __generic_file_aio_read(struct kiocb *iocb,
const struct iovec *iov,
837     unsigned long nr_segs, loff_t *ppos)
838 {
839     struct file *filp = iocb->ki_filp;
840     ssize_t retval;
841     unsigned long seg;
842     size_t count;
843
844     count = 0;
845     for (seg = 0; seg < nr_segs; seg++) {
846         const struct iovec *iv = &iov[seg];
847         ...
852         count += iv->iov_len;
853         if (unlikely((ssize_t)(count+iv->iov_len) <
0))
854             return -EINVAL;
855         if (access_ok(VERIFY_WRITE, iv->iov_base,
iv->iov_len))
856             continue;
857         if (seg == 0)
858             return -EFAULT;
859         nr_segs = seg;
860         count -= iv->iov_len;
861         break;
862     }
863     ...
-----
```

第 835~842 行：回想一下，`nr_segs` 通过我们的调用程序被设置为 1，并且 `iocb` 和 `iov` 包含了文件指针和缓冲区的信息。现在马上从 `iocb` 中提取文件指针。

第 845~862 行：这个 `for` 循环检验传入的 `iovec` 结构体是否由有效的片段组成。回忆一下，它包含了用户空间的缓冲区信息。

```
-----
mm/filemap.c
...
863
864 /* coalesce the iovecs and go direct-to-BIO for O_DIRECT */
865 if (filp->f_flags & O_DIRECT) {
866     loff_t pos = *ppos, size;
867     struct address_space *mapping;
868     struct inode *inode;
869
870     mapping = filp->f_mapping;
```

```

871     inode = mapping->host;
872     retval = 0;
873     if (!count)
874         goto out; /* skip atime */
875     size = i_size_read(inode);
876     if (pos < size) {
877         retval = generic_file_direct_IO(READ, iocb,
878             iov, pos, nr_segs);
879         if (retval >= 0 && !is_sync_kiocb(iocb))
880             retval = -EIOCBQUEUED;
881         if (retval > 0)
882             *ppos = pos + retval;
883     }
884     file_accessed(filp);
885     goto out;
886 }
...

```

第 863~886 行：仅当读操作是直接 I/O 时才进入这个代码区。直接 I/O 能够绕过页缓存，是某些块设备非常有用的特性。然而，根据我们的需求，根本不会进入这段代码。大多数文件 I/O 把我们的访问路径视为页缓存（这一点将在稍后讨论），它比底层的块设备要快得多。

```

-----
mm/filemap.c
...
887
888     retval = 0;
889     if (count) {
890         for (seg = 0; seg < nr_segs; seg++) {
891             read_descriptor_t desc;
892
893             desc.written = 0;
894             desc.buf = iov[seg].iov_base;
895             desc.count = iov[seg].iov_len;
896             if (desc.count == 0)
897                 continue;
898             desc.error = 0;
899             do_generic_file_read(filp, ppos, &desc, file_read_actor);
900             retval += desc.written;
901             if (!retval) {
902                 retval = desc.error;
903                 break;
904             }
905         }
906     }
907 out:
908     return retval;
909 }
-----

```

第 889~890 行：因为我们的 `iovec` 是有效的，并且只有一个片段，因此只执行一次这个 `for` 循环。

第 891~898 行：将 `iovec` 结构转换成 `read_descriptor_t` 结构。`read_descriptor_t` 结构记录读的状态。此处是对 `read_descriptor_t` 结构的描述：

```
-----
include/linux/fs.h
837  typedef struct {
838      size_t written;
839      size_t count;
840      char __user * buf;
841      int error;
842  } read_descriptor_t;
-----
```

第 838 行：字段 `written` 存放不断变化着的已传送字节数。

第 839 行：字段 `count` 存放不断变化着的未传送字节数。

第 840 行：字段 `buf` 存放缓冲区中的当前位置。

第 841 行：字段 `error` 存放读操作期间遇到的任何错误码。

第 899 行：把新的 `read_descriptor_t` 结构 **desc** 连同文件指针 `filp` 和位置 `ppos` 一起传给 `do_generic_file_read()`。`file_read_actor()`^①是一个函数，它把一个页面复制到由 `desc` 指定的用户空间缓冲区。

第 900~909 行：计算已读字节数，并把它返回给调用者。

这里进入了 `read()` 的内部，我们准备访问页缓存^②，然后确定我们想要读的文件区是否已经在 RAM 中，如果在，就不必直接访问块设备。

2. 追踪页缓存

回想一下我们遇到的最后一个函数，它给 `do_generic_file_read()` 传递了文件指针 `filp`、偏移量 `ppos`、`read_descriptor_t` 类型的 `desc`，以及函数 `file_read_actor` 这些参数。

```
-----
include/linux/fs.h
1420 static inline void do_generic_file_read(struct file * filp, loff_t *ppos,
1421      read_descriptor_t * desc,
1422      read_actor_t actor)
1423 {
1424     do_generic_mapping_read(filp->f_mapping,
1425         &filp->f_ra,
1426         filp,
1427         ppos,
1428         desc,
1429         actor);
1430 }
-----
```

第 1420~1430 行：`do_generic_file_read()` 只不过是对 `do_generic_mapping_read()` 的封装。`filp->f_mapping` 是指向 `address_space` 对象的指针，`filp->f_ra` 是一个存放文件预读

① `file_read_actor()` 参见 `mm/filemap.c` 的第 794 行。

② 6.4 节中描述了页缓存。

状态地址的结构^①。

因此，我们可以通过文件指针中的 `address_space` 对象，把对文件的读转换成对页缓存的读。由于 `do_generic_mapping_read()` 是一个非常长的函数，带有许多分支情况，因此，我们设法尽可能简单地对这些代码进行分析。

```
-----
mm/filemap.c
645 void do_generic_mapping_read(struct address_space *mapping,
646     struct file_ra_state *_ra,
647     struct file * filp,
648     loff_t *ppos,
649     read_descriptor_t * desc,
650     read_actor_t actor)
651 {
652     struct inode *inode = mapping->host;
653     unsigned long index, offset;
654     struct page *cached_page;
655     int error;
656     struct file_ra_state ra = *_ra;
657
658     cached_page = NULL;
659     index = *ppos >> PAGE_CACHE_SHIFT;
660     offset = *ppos & ~PAGE_CACHE_MASK;
-----
```

第 652 行：从 `address_space` 提取正在读的文件的索引节点。

第 658~660 行：把 `cached_page` 初始化成 `NULL`，直到可以确定它是否在页缓存中。同时计算基于页缓存约束的 `index` 和 `offset`。`index` 对应页缓存中的页号，而 `offset` 对应页内偏移。当页大小是 4 096 字节时，对文件指针右移 12 位就得到了页的索引。

“页缓存可以在多于一页的大块内完成，因为这样可获得更高的吞吐量”（`linux/pagemap.h`）。`PAGE_CACHE_SHIFT` 和 `PAGE_CACHE_MASK` 是控制页缓存的结构和页缓存大小的宏：

```
-----
mm/filemap.c
661
662     for (;;) {
663         struct page *page;
664         unsigned long end_index, nr, ret;
665         loff_t isize = i_size_read(inode);
666
667         end_index = isize >> PAGE_CACHE_SHIFT;
668
669         if (index > end_index)
670             break;
671         nr = PAGE_CACHE_SIZE;
672         if (index == end_index) {
-----
```

^① 关于这个字段以及预读优化的更多信息参见“文件结构”部分。

```

673     nr = isize & ~PAGE_CACHE_MASK;
674     if (nr <= offset)
675         break;
676 }
677
678 cond_resched();
679 page_cache_readahead(mapping, &ra, filp, index);
680
681 nr = nr - offset;
-----

```

第 662~681 行：这段代码在页缓存上反复循环，获取足够的页以得到读命令请求的字节。

```

-----
mm/filemap.c
682 find_page:
683     page = find_get_page(mapping, index);
684     if (unlikely(page == NULL)) {
685         handle_ra_miss(mapping, &ra, index);
686         goto no_cached_page;
687     }
688     if (!PageUptodate(page))
689         goto page_not_up_to_date;
-----

```

第 682~689 行：我们试图找到第一个被请求的页。如果这个页不在页缓存中，就跳转到标号 no_cached_page 处。如果该页不是最新的，就跳转到标号 page_not_up_to_date 处。find_get_page() 使用地址空间的基树来查找索引为 index 的页，其中 index 是指定的偏移量。

```

-----
mm/filemap.c
690 page_ok:
691     /* If users can be writing to this page using arbitrary
692     * virtual addresses, take care about potential aliasing
693     * before reading the page on the kernel side.
694     */
695     if (mapping_writably_mapped(mapping))
696         flush_dcache_page(page);
697
698     /*
699     * Mark the page accessed if we read the beginning.
700     */
701     if (!offset)
702         mark_page_accessed(page);
703     ...
714     ret = actor(desc, page, offset, nr);
715     offset += ret;
716     index += offset >> PAGE_CACHE_SHIFT;
717     offset &= ~PAGE_CACHE_MASK;
718
719     page_cache_release(page);
720     if (ret == nr && desc->count)

```

```

721     continue;
722     break;
723

```

第 690~723 行：这些内嵌的注释已给出详细描述，因此没有必要赘述。请注意第 656 行~658 行，如果要获取更多的页，就立刻返回到循环的开始，在那里，第 714~716 行对 `index` 和 `offset` 的处理有助于选择下一个要获取的页。如果没有更多的页要读，就跳出 `for` 循环。

```

-----
mm/filemap.c
724 page_not_up_to_date:
725     /* Get exclusive access to the page ... */
726     lock_page(page);
727
728     /* Did it get unhashed before we got the lock? */
729     if (!page->mapping) {
730         unlock_page(page);
731         page_cache_release(page);
732         continue;
733     }
734
735     /* Did somebody else fill it already? */
736     if (PageUptodate(page)) {
737         unlock_page(page);
738         goto page_ok;
739     }
740

```

第 724~740 行：如果该页不是最新的，就再检查一次；如果该页现在是最新的，就立刻返回到标号 `page_ok` 处。否则，将设法获取对该页的独占访问；这将导致我们睡眠，直到获得对该页的独占访问。获得独占访问后，就来看看这个页是否企图从页缓存中删除自己，如果是，在返回到 `for` 循环顶部前赶紧继续向前。如果它仍然存在并且现在是最新的，就对页解锁并跳转到标号 `page_ok` 处。

```

-----
mm/filemap.c
741 readpage:
742     /* ... and start the actual read. The read will unlock the page. */
743     error = mapping->a_ops->readpage(filp, page);
744
745     if (!error) {
746         if (PageUptodate(page))
747             goto page_ok;
748         wait_on_page_locked(page);
749         if (PageUptodate(page))
750             goto page_ok;
751         error = -EIO;
752     }
753

```

```

754     /* UHHUH! A synchronous read error occurred. Report it */
755     desc->error = error;
756     page_cache_release(page);
757     break;
758
-----

```

第 741~743 行：如果该页不是最新的，就跳到前一个标号 `page_not_up_to_date` 处，并对该页加锁。实际的读操作 `mapping->a_ops->readpage(filp, page)` 对该页进行解锁（我们将会稍微深入地对 `readpage()` 进行追踪，但是让我们首先完成当前的解释）。

第 746~750 行：如果成功地读取了一个页，检查它是否是最新的，如果是则跳转到标号 `page_ok` 处。

第 751~758 行：如果发生同步读错误，就在 `desc` 中记录这个错误，同时从页缓存释放该页，并跳出 `for` 循环。

```

-----
mm/filemap.c
759 no_cached_page:
760     /*
761     * Ok, it wasn't cached, so we need to create a new
762     * page..
763     */
764     if (!cached_page) {
765         cached_page = page_cache_alloc_cold(mapping);
766         if (!cached_page) {
767             desc->error = -ENOMEM;
768             break;
769         }
770     }
771     error = add_to_page_cache_lru(cached_page, mapping,
772                                 index, GFP_KERNEL);
773     if (error) {
774         if (error == -EEXIST)
775             goto find_page;
776         desc->error = error;
777         break;
778     }
779     page = cached_page;
780     cached_page = NULL;
781     goto readpage;
782 }
-----

```

第 768~772 行：如果要读的页没有被缓存，就在地址空间中分配一个新的页，并把它添加到最近最少使用（LRU）缓存和页缓存中。

第 773~775 行：向缓存添加页时，如果因为页已经存在而产生错误，就跳转到标号 `find_page` 处并再试一次。如果多个进程试图读取同一个未缓冲的页，也可能发生这种情况；一个进程试图分配页并且分配成功的话，另一个进程试图再分配时就会发现该页已经存在。

第 776~777 行：如果向缓存添加页时出现的错误并不是页已存在的错误，则记录该错误并跳

出 for 循环。

第 779~781 行：当我们成功地分配了页并将页加入页缓存和 LRU 缓存后，就让指针 page 指向新页，并通过跳转到标号 readpage 处准备开始读取该页。

```
-----
mm/filemap.c
784  *_ra = ra;
785
786  *ppos = ((loff_t) index << PAGE_CACHE_SHIFT) + offset;
787  if (cached_page)
788      page_cache_release(cached_page);
789  file_accessed(filp);
790 }
```

第 786 行：计算基于页缓存索引和偏移量的实际偏移量。

第 787~788 行：如果分配了一个新页，并且能够把它正确地添加到页缓存中的话，就删除这个页。

第 789 行：通过索引节点更新文件的最后一次访问时间。

这个函数中描述的逻辑是页缓存的核心。注意，页缓存如何避免触及任何特定文件系统的数据。这使得 Linux 内核不用考虑底层文件系统的结构，用页缓存就可以缓存各种各样的页。因此，页缓存能够同时容纳来自 MINIX、ext2 以及 MSDOS 的页。

通过调用地址空间的 readpage() 函数，页缓存维护着特定文件系统层之间的差异。每个特定的文件系统都需要实现自己的 readpage()。因此，当通用文件系统层调用 mapping->a_ops->readpage() 时，它从文件系统驱动程序的 address_space_operations 结构中调用具体的 readpage() 函数，对于 ext2 文件系统而言，readpage() 定义如下：

```
-----
fs/ext2/inode.c
676 struct address_space_operations ext2_aops = {
677     .readpage      = ext2_readpage,
678     .readpages     = ext2_readpages,
679     .writepage     = ext2_writepage,
680     .sync_page     = block_sync_page,
681     .prepare_write = ext2_prepare_write,
682     .commit_write  = generic_commit_write,
683     .bmap          = ext2_bmap,
684     .direct_IO     = ext2_direct_IO,
685     .writepages    = ext2_writepages,
686 };
```

因此，readpage() 实际上调用了 ext2_readpage()：

```
-----
fs/ext2/inode.c
616 static int ext2_readpage(struct file *file, struct page *page)
617 {
```



```

618     return mpage_readpage(page, ext2_get_block);
619 }

```

ext2_readpage()调用通用文件系统层的 mpage_readpage(),但会给后者传入特定文件系统层的函数 ext2_get_block()。

通用文件系统函数 mpage_readpage()把 get_block()函数作为它的第二个参数。每个文件系统都实现某些 I/O 函数,这些函数专门对应于这种文件系统的格式;get_block()便是这些函数中的一个。文件系统的 get_block()函数将 address_space 页中的逻辑块映射为特定文件系统布局中实际的设备块。让我们来分析 mpage_readpage()的细节:

```

-----
fs/mpage.c
358 int mpage_readpage(struct page *page, get_block_t get_block)
359 {
360     struct bio *bio = NULL;
361     sector_t last_block_in_bio = 0;
362
363     bio = do_mpage_readpage(bio, page, 1,
364         &last_block_in_bio, get_block);
365     if (bio)
366         mpage_bio_submit(READ, bio);
367     return 0;
368 }
-----

```

第 360~361 行:为了管理地址空间所使用的 bio 结构,我们为其分配空间以管理正试图从设备读取的页。

第 363~364 行:调用 do_mpage_readpage()把逻辑页转换成由实际的页和块组成的 bio 结构。bio 结构记录着与块 I/O 相关的信息。

第 365~367 行:将新创建的 bio 结构发送到 mpage_bio_submit()并返回。

让我们花点时间,从宏观上扼要地阐述一下迄今为止 read 函数的流程。

- ❑ 利用来自 open()调用的文件描述符定位文件指针,并从文件指针获得索引节点。
- ❑ 文件系统层在内存的页缓存中检查与给定索引节点对应的一个或多个页。
- ❑ 如果在页缓存中找不到所需的页,文件系统层使用特定文件系统的驱动程序把所请求的文件片断转换成特定设备上的 I/O 块。
- ❑ 在页缓存的 address_space 中为页分配空间,并建立一个 bio 结构,该结构把新分配的页与块设备上的扇区对应起来。

mpage_readpage()是建立 bio 结构的函数,它把从页缓存中新分配的页和 bio 结构联系起来。不过,此时页中还没有数据。因此,文件系统层需要块设备的驱动程序来完成到设备的实际接口。这是由 mpage_bio_submit()中的 submit_bio()函数来完成的:

```

-----
fs/mpage.c
90 struct bio *mpage_bio_submit(int rw, struct bio *bio)
91 {

```

```

92  bio->bi_end_io = mpage_end_io_read;
93  if (rw == WRITE)
94      bio->bi_end_io = mpage_end_io_write;
95  submit_bio(rw, bio);
96  return NULL;
97  }

```

第 90 行：首先要注意的是：通过传递 `rw` 参数，`mpage_bio_submit()` 既可以为 `read` 调用服务，又可以为 `write` 调用服务。它提交一个 `bio` 结构，在 `read` 调用的情况下，这个结构是空的，所以必须被填充。在 `write` 调用的情况下，`bio` 结构是已经填充好的，块设备驱动程序把其中的内容复制到自己的设备中。

第 92~94 行：如果正在读或正在写页面，就设置合适的函数作为 I/O 结束时调用的函数。

第 95~96 行：调用 `submit_bio()` 并返回 `NULL`。回想一下，`mpage_readpage()` 对 `mpage_bio_submit()` 的返回值不做任何处理。

`submit_bio()` 是 Linux 内核通用块设备驱动层的组成部分。

```

-----
drivers/block/ll_rw_blk.c
2433 void submit_bio(int rw, struct bio *bio)
2434 {
2435     int count = bio_sectors(bio);
2436
2437     BIO_BUG_ON(!bio->bi_size);
2438     BIO_BUG_ON(!bio->bi_io_vec);
2439     bio->bi_rw = rw;
2440     if (rw & WRITE)
2441         mod_page_state(pgpgout, count);
2442     else
2443         mod_page_state(pgpgin, count);
2444
2445     if (unlikely(block_dump)) {
2446         char b[BDEVNAME_SIZE];
2447         printk(KERN_DEBUG "%s(%d): %s block %Lu on %s\n",
2448             current->comm, current->pid,
2449             (rw & WRITE) ? "WRITE" : "READ",
2450             (unsigned long long)bio->bi_sector,
2451             bdevname(bio->bi_bdev, b));
2452     }
2453
2454     generic_make_request(bio);
2455 }

```

第 2433~2443 行：这几个调用启用一些调试：设置 `bio` 结构的读/写属性，执行部分页状态的管理工作。

第 2445~2452 行：这几行处理偶然发生块转储时的情况。它将抛出一个调试消息。

第 2454 行：`generic_make_request()` 包含主要功能，并利用特定块设备驱动程序的请求队列来处理块的 I/O 操作。

下面是 generic_make_request() 部分内嵌的注释：

```
-----
drivers/block/ll_rw_blk.c
2336 * The caller of generic_make_request must make sure that bi_io_vec
2337 * are set to describe the memory buffer, and that bi_dev and bi_sector
are
2338 * set to describe the device address, and the
2339 * bi_end_io and optionally bi_private are set to describe how
2340 * completion notification should be signaled.
-----
```

在以上这些阶段，我们构造了 bio 结构，因此，bio_vec 结构被映射到第 2337 行提到的内存缓冲区，bio 结构也用设备地址参数来初始化。如果想要深入到块设备的驱动程序继续研究读操作，请参考 5.2.1 节，它描述了块设备的驱动程序如何处理请求队列以及设备的具体硬件约束条件。图 6-18 说明了 read() 系统调用是如何穿越内核的各个功能层的。

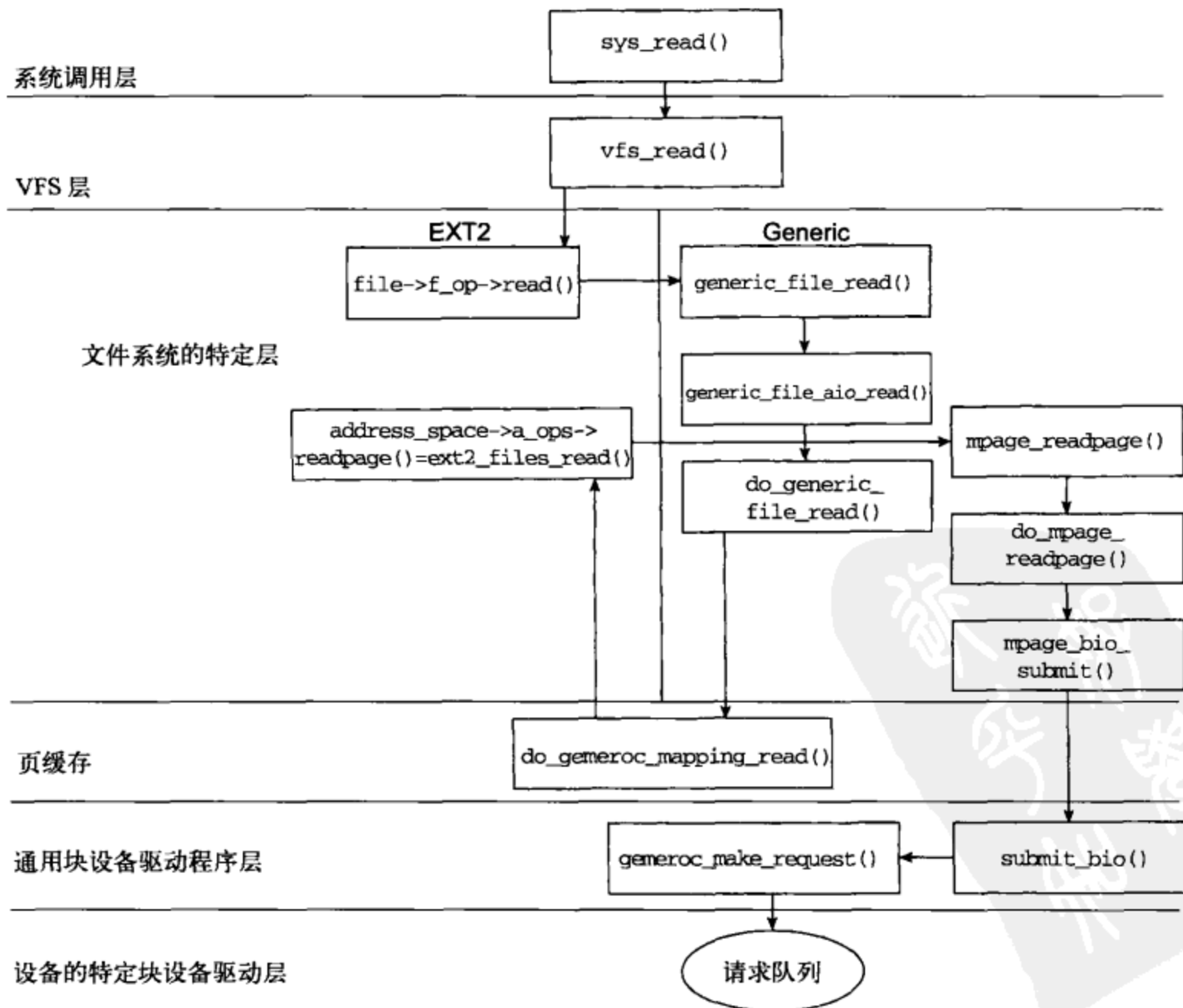


图 6-18 read() 自顶而下的遍历

块设备的驱动程序读取实际的数据并把它们放入 bio 结构以后，我们追踪的代码就展开了。页缓存中新分配的页被填充，页的引用被传回 VFS 层，并把页中的数据复制到指定的用户空间区域——这就是最初由 read() 调用给出的缓冲区。

也许读者会问：“难道只介绍一半的内容吗？如果我们想要写，而不是读呢？”

我们希望以上描述给出一条清晰的思路，read() 调用在整个 Linux 内核中穿越的轨迹与 write() 调用类似，但还是要概述一下二者的差异。

6.5.4 write()

write() 调用被映射到 sys_write()，而 sys_write() 被映射 vfs_write()，与 read() 调用的映射方式完全相同：

```
-----
fs/read_write.c
244 ssize_t vfs_write(struct file *file, const char __user *buf, size_t count, loff_t *pos)
245 {
...
259     ret = file->f_op->write(file, buf, count, pos);
...
268 }
-----
```

vfs_write() 使用 file_operations 的通用 write 函数，以确定使用哪一个具体文件系统层的 write。在我们举的例子 ext2 中，可以通过 ext2_file_operations 结构进行转换：

```
-----
fs/ext2/file.c
42 struct file_operations ext2_file_operations = {
43     .llseek    = generic_file_llseek,
44     .read      = generic_file_read,
45     .write     = generic_file_write,
...
56 };
-----
```

第 44~45 行：此处调用 generic_file_write()，而不是 generic_file_read()。

generic_file_write() 获得文件的锁，以防止两个写程序同时写一个文件，并调用 generic_file_write_nolock()。后者把文件指针和缓冲区分别转换成 kiocb 参数和 iovec 参数，最后调用页缓存的写函数 generic_file_aio_write_nolock()。

此处是写和读分道扬镳的地方。如果要写的页不在页缓存中，对设备本身而言，写操作并不是以失败告终。相反，它把页读入页缓存，然后再执行写操作。页缓存中的页不会被立刻写到磁盘上，而是它们被标记为“脏”，所有的脏页都要被定期地写回磁盘。

这是与 read() 函数的 readpage() 类似的函数。在 generic_file_aio_write_nolock() 内部，通过 address_space_operations 的指针访问 prepare_write() 和 commit_write()，这两个函数是文件所驻留的文件系统类型所特有的。回想一下，ext2_aops，我们会看到 ext2 的驱动程序使用它自己的函数 ext2_prepare_write() 和通用函数 generic_commit_write()。

```

-----
fs/ext2/inode.c
628 static int
629 ext2_prepare_write(struct file *file, struct page *page,
630     unsigned from, unsigned to)
631 {
632     return block_prepare_write(page, from, to, ext2_get_block);
633 }
-----

```

第 632 行: `ext2_prepare_write` 只不过是对通用文件系统函数 `block_prepare_write()` 的简单封装, 它将传入 `ext2` 文件系统所特有的函数 `get_block()`。

`block_prepare_write()` 分配写操作所需的新缓冲区。例如, 如果要把数据添加到文件末尾, 就要创建足够的缓冲区, 并把它们和页关联起来以存放新数据。

`generic_commit_write()` 获得给定的页, 并扫描页内的缓冲区, 标记每个脏的页。把写操作分解成准备和提交两步以防止把不完整的写数据从页缓存刷新到块设备。

刷出脏页

`write()` 把已经写过的所有页插入页缓存并标记为脏之后返回。Linux 有一个后台程序 `pdflush`, 它能够在以下两种情况下把脏页从页缓存写到块设备。

- 系统的空闲内存低于阈值。刷出页缓存中的页以释放内存。
- 脏页达到了某个年龄。把某段时间后仍没有写回磁盘的页写到它们的块设备中。

当准备好向磁盘写页时, 后台程序 `pdflush` 将调用特定于文件系统的 `writpages()` 函数。因此, 在我们的例子中, 回想一下 `ext2_file_operation` 结构, 在该结构中, `writpages()` 就等同于 `ext2_writpages()`^①。

```

-----
670 static int
671 ext2_writpages(struct address_space *mapping, struct writeback_control *wbc)
672 {
673     return mpage_writpages(mapping, wbc, ext2_get_block);
674 }
-----

```

与其他通用文件系统函数的具体实现类似, `ext2_writpages()` 只不过是调用了通用文件系统的函数 `mpage_writpages()`, 并把特定于文件系统的函数 `ext2_get_block()` 作为第二参数传递给它。

`mpage_writpages()` 扫描脏页, 并对每个脏页调用 `mpage_writpage()`。与 `mpage_readpage()` 类似, `mpage_writpage()` 返回一个 `bio` 结构, 该结构把页的物理设备布局映射成其物理内存布局。`mpage_writpages()` 接着调用 `submit_bio()` 向块设备的驱动程序发送新创建的 `bio` 结构, 从而把数据传送到设备本身。

① `Pdflush` 后台例程是相当棘手的, 我们的目的是追踪 `write`, 因此忽略其复杂性。然而, 如果你对细节感兴趣的话, 文件 `mm/pdflush.c`、`mm/fs-writeback.c` 和 `mm/page-writeback.c` 中包含了相关代码。

6.6 小结

本章首先分析了通用文件模型所涉及的结构和全局变量，这些结构包括超级块、索引节点、目录项以及文件结构。然后研究了 VFS 的相关结构，并讨论了 VFS 支持各种文件系统的工作机理。

接着，研究了与 VFS 相关的系统调用 `open` 和 `close`，分析了它们之间如何协同工作。然后，追踪了 `read()` 和 `write()` 这两个用户空间的调用——从 VFS 穿越到底层，涵盖了通用文件系统层和特定文件系统层之间的细节。为了对特定文件系统层加以说明，我们以 `ext2` 文件系统的驱动程序为例，分析了内核是如何穿插调用特定于文件系统的驱动程序函数和通用文件系统的函数。这使得我们讨论了页缓存，页缓存是一片内存区域，它存放的最近访问的页是从连接到系统的块设备读取的。

6.7 习题

1. 在什么情况下，你会使用索引节点的 `i_hash` 字段而不是 `i_list` 字段？为什么同一个结构中既有散列表又有线性表？
2. 在我们遇到的所有文件结构中，指出在硬盘上有相应数据结构的结构。
3. 目录项对象用于什么类型的操作？为什么不只用索引节点？
4. 文件描述符和文件结构之间有什么联系？是一对一，多对一，还是一对多？
5. `fd_set` 结构的用途是什么？
6. 什么类型的数据结构保证了页缓存以最大的速度操作？
7. 假如你正在写一个新的文件系统驱动程序。你要用新驱动程序 (`media_fs`) (可以优化多媒体的文件 I/O) 替换 `ext2` 文件系统驱动程序。你会在哪里对 Linux 内核进行修改，以确保用的是你的新驱动程序，而不是 `ext2` 的驱动程序？
8. 页怎样变脏？如何把脏页写回磁盘？



进程调度和内核同步

本章内容

- Linux 的调度程序
- 内核抢占
- 自旋锁和信号量
- 系统时钟：关于时间和定时器

Linux 内核是多任务内核，这意味着多个进程可以同时运行，就好像它们是系统中唯一的进程一样。操作系统在特定时刻选择哪一个进程访问系统的 CPU 是由调度程序来决定的。

调度程序负责在不同的进程之间切换 CPU，并决定进程访问 CPU 的次序。与大多数操作系统一样，Linux 通过时钟中断来触发调度程序。当时钟中断发生时，内核必须决定是否为其他非当前进程让出 CPU，如果能让出的话，接下来应该由哪一个进程获得 CPU。相邻两个时钟中断之间的时间间隔称为时间片（timeslice）。

系统中的进程往往分成两种类型：交互式的和非交互式的。交互式进程非常依赖于 I/O，因此，它们通常用不尽自己的时间片，而是将 CPU 让给其他进程。

非交互式进程则非常依赖于 CPU，它们通常都会花掉大部分（如果不是全部的话）时间片。调度程序必须平衡这两类进程的需求，努力确保每个进程都能获得足够多的时间去完成任务，而不会对其他进程的执行产生不利的影响。

与某些调度程序相似，Linux 还要区别对待另一类进程：实时进程。实时进程必须实时地执行。Linux 支持实时进程，但是它们是处于调度逻辑之外的。简单地说，Linux 的调度程序认为，标记为实时的所有进程的优先级比其他任何进程都高。开发者必须确保实时进程不会贪婪地占有 CPU，并确保它们最终会放弃 CPU。

调度程序通常使用某种类型的进程队列来管理系统中进程的执行。在 Linux 中，这个进程队列称为运行队列^①。在第 3 章中完整地描述了运行队列，但是由于运行队列与调度程序关系密切，这里再次简要阐述一下它的基本原理。

在 Linux 中，运行队列由两个优先级数组组成。

- 活跃数组：存放时间片还没有耗尽的进程。

^① 3.6 节介绍了运行队列。

□ 到期数组：存放时间片已经耗尽的进程。

总的来说，Linux 中调度程序的工作就是：找出优先级最高的活跃进程，允许它们使用 CPU 并投入运行；当进程耗尽自己的时间片时把它放入到期数组。理解了这个总的框架后，再来详细分析 Linux 的调度程序究竟是如何工作的。

7.1 Linux 的调度程序

Linux 2.6 内核引入了一个全新的调度程序，一般称为 $O(1)$ 调度程序，它能够在恒定的时间内完成进程的调度^①。第 3 章讨论了调度程序的基本结构，以及如何初始化新创建的进程，使之参与调度。本节首先描述进程如何在一个单 CPU 系统上执行，当然也会提及多 CPU (SMP) 系统中的调度代码。但是，同一个调度过程通常适用于不同的 CPU。接下来描述调度程序如何通过上下文的切换来换出当前正在运行的进程，最后将简单介绍 2.6 内核中的另一个重大变化：内核抢占。

总的来说，调度程序只不过是对给定数据结构进行操作的一组函数。实现调度程序的所有代码几乎都能在文件 `kernel/sched.c` 和 `include/linux/sched.h` 中找到。要再次强调的一点是：调度程序的代码交替地使用术语“任务”和“进程”。有时候，代码注释中也用“线程”来指代任务或进程。在调度程序中，任务（或进程）是数据结构和控制流的集合。调度程序的代码也引用 `task_struct` 这个 Linux 内核用来记录进程信息的数据结构^②。

7.1.1 选择下一个进程

进程被初始化并放到运行队列后，它应该在某个时刻获得 CPU 的控制权并投入运行。函数 `schedule()` 和 `scheduler_tick()` 负责把 CPU 的控制权传递给不同的进程。`scheduler_tick()` 是一个由内核周期性调用的系统定时器，它把进程标记为需要重新调度。当定时器事件发生时，当前进程就被暂停，Linux 内核本身接管对 CPU 的控制权。待定时器事件结束后，Linux 内核通常把控制权交回刚刚被暂停的进程。然而，当这个进程被标记为需要重新调度时，内核调用 `schedule()` 来选择将要激活哪个进程，当然，并不是在内核接管 CPU 控制权之前正在执行的那个进程。我们把在内核接管控制权之前正在执行的进程称为当前进程 (current process)。在某些稍微复杂一些的情况中，内核可以从内核本身获得控制权，这称为内核抢占 (kernel preemption)。为简单起见，在接下来的几节中，假定调度程序只需要决定究竟选择用户空间两个进程中的哪一个来获得 CPU 的控制权。

图 7-1 说明了随着时间的推移，如何在各个进程之间传递 CPU 的控制权。如图所示，进程 A 拥有对 CPU 的控制权并且正在执行。系统定时器 `scheduler_tick()` 执行后，将从 A 获得对 CPU 的控制权，并把 A 标记为需要重新调度。Linux 内核调用 `schedule()`，调度程序选择了进程 B，这样，CPU 的控制权就交给了进程 B。

① $O(1)$ 是大 O 表示法，它意味着调度程序的开销是恒定的，与系统当前的负载无关。

② 第 3 章详细介绍了 `task_struct` 结构。

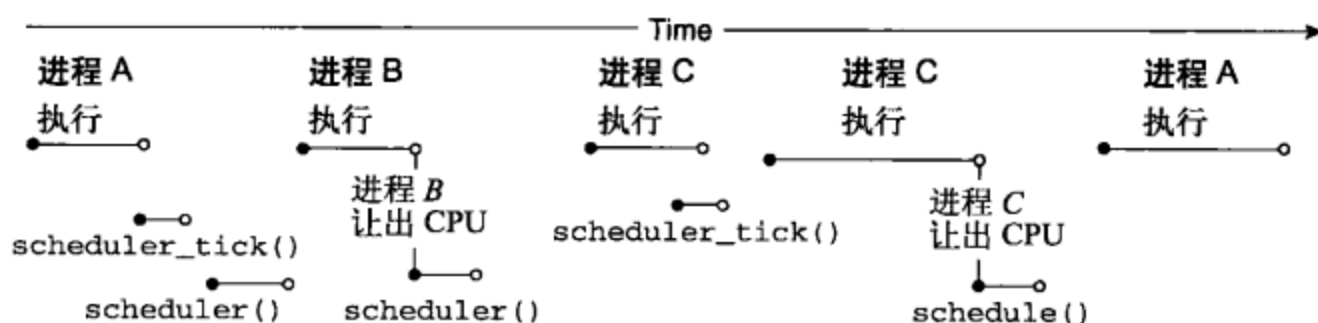


图 7-1 调度进程

进程 B 执行了一会儿后主动让出了 CPU。这种情况通常发生在进程等待资源的时候。进程 B 调用了 `schedule()`，调度程序选择进程 C 来执行。

进程 C 开始执行，直到 `scheduler_tick()` 发生，`scheduler_tick()` 没有把 C 标记为需要重新调度。显然，`schedule()` 不被调用，因此，C 再次获得 CPU 的控制权。

通过调用 `schedule()`，进程 C 让出了 CPU，`schedule()` 决定进程 A 应该获得 CPU 的控制权，于是 A 再次开始执行。

`schedule()` 决定接下来要执行的是哪个进程，`scheduler_tick()` 决定哪个进程必须让出 CPU。此处，首先分析 `schedule()`，然后分析 `scheduler_tick()`。这两个函数一起完美地展示了调度程序的控制流程。

```
-----
kernel/sched.c
2184 asmlinkage void schedule(void)
2185 {
2186     long *switch_count;
2187     task_t *prev, *next;
2188     runqueue_t *rq;
2189     prio_array_t *array;
2190     struct list_head *queue;
2191     unsigned long long now;
2192     unsigned long run_time;
2193     int idx;
2194
2195     /*
2196     * Test if we are atomic. Since do_exit() needs to call into
2197     * schedule() atomically, we ignore that path for now.
2198     * Otherwise, whine if we are scheduling when we should not be.
2199     */
2200     if (likely(!(current->state & (TASK_DEAD | TASK_ZOMBIE)))) {
2201         if (unlikely(in_atomic())) {
2202             printk(KERN_ERR "bad: scheduling while atomic!\n");
2203             dump_stack();
2204         }
2205     }
2206
2207     need_resched:
2208     preempt_disable();
2209     prev = current;
2210     rq = this_rq();
2211
2212     release_kernel_lock(prev);
```

```

2213  now = sched_clock();
2214  if (likely(now - prev->timestamp < NS_MAX_SLEEP_AVG))
2215      run_time = now - prev->timestamp;
2216  else
2217      run_time = NS_MAX_SLEEP_AVG;
2218
2219  /*
2220   * Tasks with interactive credits get charged less run_time
2221   * at high sleep_avg to delay them losing their interactive
2222   * status
2223   */
2224  if (HIGH_CREDIT(prev))
2225      run_time /= (CURRENT_BONUS(prev) ? : 1);
-----

```

第 2213~2218 行：计算进程被调度程序激活后运行了多长时间。如果进程被激活的时间长度大于最大平均睡眠时间（NS_MAX_SLEEP_AVG），则将其运行时间设置为最大平均睡眠时间。

这就是 Linux 内核在其他代码片段中所谓的时间片。时间片既指调度程序中断的间隔时间，又指进程使用 CPU 的时间长度。如果进程耗尽了时间片，它就到期了，不再是活跃进程。时间戳是一个用来确定进程使用 CPU 的时间长度的绝对值。调度程序利用时间戳来减小已使用过 CPU 的进程的时间片。

例如，假定进程 A 有一个 50 个时钟周期的时间片。它使用了 5 个时钟周期的 CPU，然后为其他进程让出 CPU。内核利用时间戳便可确定进程 A 的时间片还剩 45 个周期。

第 2224~2225 行：交互式进程是那些花费大量时间用于等待输入的进程。键盘控制器就是一个很好的交互式进程的例子，它大部分时间都在等待输入，但是当它有任务要执行时，用户又希望它拥有高优先级。

交互式进程，指那些交互信用超过 100（默认值）的进程，它们的有效 run_time 被 (sleep_avg/max_sleep_avg*MAX_BONUS(10)) 除^①：

```

-----
kernel/sched.c
2226
2227  spin_lock_irq(&rq->lock);
2228
2229  /*
2230   * if entering off of a kernel preemption go straight
2231   * to picking the next task.
2232   */
2233  switch_count = &prev->nivcs;
2234  if (prev->state && !(preempt_count() & PREEMPT_ACTIVE)) {
2235      switch_count = &prev->nvcsw;
2236      if (unlikely((prev->state & TASK_INTERRUPTIBLE) &&
2237                  unlikely(signal_pending(prev))))
2238          prev->state = TASK_RUNNING;
2239      else
2240          deactivate_task(prev, rq);
2241  }
-----

```

① BONUS 是对高优先级进程的调度奖励。

第 2227 行：该函数获得即将要被修改的运行队列的锁。

第 2233~2241 行：如果由于内核抢占前一个进程而进入了 `schedule()`，那么，若该进程有挂起的信号，就让前一个进程有继续运行的资格，这就意味着内核紧接着抢占了正常的处理；因此，代码包含了两个 `unlikely()` 语句^①。如果接下来没有继续发生内核抢占的话，就从运行队列删除被抢占的进程并继续选择下一个要运行的进程。

```
-----
kernel/sched.c
2243  cpu = smp_processor_id();
2244  if (unlikely(!rq->nr_running)) {
2245      idle_balance(cpu, rq);
2246      if (!rq->nr_running) {
2247          next = rq->idle;
2248          rq->expired_timestamp = 0;
2249          wake_sleeping_dependent(cpu, rq);
2250          goto switch_tasks;
2251      }
2252  }
2253
2254  array = rq->active;
2255  if (unlikely(!array->nr_active)) {
2256      /*
2257       * Switch the active and expired arrays.
2258       */
2259      rq->active = rq->expired;
2260      rq->expired = array;
2261      array = rq->active;
2262      rq->expired_timestamp = 0;
2263      rq->best_expired_prio = MAX_PRIO;
2264  }
-----
```

第 2243 行：利用 `smp_processor_id()` 来获取当前 CPU 的标识符。

第 2244~2252 行：如果运行队列中没有进程，就将空闲进程设置为下一个要执行的进程，并且把运行队列的到期时间戳重置为 0。在多处理器系统中，在这之前还要先检查在其他 CPU 上是否有本 CPU 可以获取的进程在运行。实际上，在系统中的所有 CPU 上都加载了用于平衡负载的空闲进程。仅当没有进程能够从其他 CPU 移出时，才把空闲设置为运行队列的下一个进程并重置到期的时间戳。

第 2255~2264 行：如果运行队列的活跃数组为空，就交换活跃数组和到期数组的指针，然后选择一个新进程来运行。

```
-----
kernel/sched.c
2266  idx = sched_find_first_bit(array->bitmap);
2267  queue = array->queue + idx;
2268  next = list_entry(queue->next, task_t, run_list);
2269
```

① 关于 `unlikely` 例程的更多信息，请参阅第 2 章。

```

2270  if (dependent_sleeper(cpu, rq, next)) {
2271      next = rq->idle;
2272      goto switch_tasks;
2273  }
2274
2275  if (!rt_task(next) && next->activated > 0) {
2276      unsigned long long delta = now - next->timestamp;
2277
2278      if (next->activated == 1)
2279          delta = delta * (ON_RUNQUEUE_WEIGHT * 128 / 100) / 128;
2280
2281      array = next->array;
2282      dequeue_task(next, array);
2283      recalc_task_prio(next, next->timestamp + delta);
2284      enqueue_task(next, array);
2285  }
next->activated = 0;
-----

```

第 2266~2268 行：调度程序利用 `sched_find_first_bit()` 找到优先级最高的进程来运行，然后，让变量 `queue` 指向优先级数组的某个元素中存放的链表。最后，把变量 `next` 初始化为指向链表 `queue` 中的第一个进程。

第 2270~2273 行：如果将要被激活的进程必需等待正在睡眠的兄弟进程，那么就激活一个新的进程，并且跳转到 `switch_tasks` 处继续执行调度函数。

假设有一个进程 A，它派生出进程 B，用于从设备读取数据，此时进程 A 必须等待进程 B 完成后才能继续执行。如果此时调度程序选择激活进程 A，函数 `dependent_sleeper()` 就会发现进程 A 正在等待进程 B，于是调度程序将选择激活一个新的进程。

第 2275~2285 行：如果进程 `next` 的 `activated` 属性大于 0 并且 `next` 不是实时进程，就从链表 `queue` 中删除 `next` 进程，并在重新计算其优先级后将它再次放入队列 `queue` 中。

第 2286 行：将进程 `next` 的 `activated` 属性设置为 0，随后，`next` 以这个属性值运行。

```

-----
kernel/sched.c
2287 switch_tasks:
2288     prefetch(next);
2289     clear_tsk_need_resched(prev);
2290     RCU_qsctr(task_cpu(prev))++;
2291
2292     prev->sleep_avg -= run_time;
2293     if ((long)prev->sleep_avg <= 0) {
2294         prev->sleep_avg = 0;
2295         if (!(HIGH_CREDIT(prev) || LOW_CREDIT(prev)))
2296             prev->interactive_credit--;
2297     }
2298     prev->timestamp = now;
2299
2300     if (likely(prev != next)) {
2301         next->timestamp = now;
2302         rq->nr_switches++;
2303         rq->curr = next;

```

```

2304     ++*switch_count;
2305
2306     prepare_arch_switch(rq, next);
2307     prev = context_switch(rq, prev, next);
2308     barrier();
2309
2310     finish_task_switch(prev);
2311 } else
2312     spin_unlock_irq(&rq->lock);
2313
2314     reacquire_kernel_lock(current);
2315     preempt_enable_no_resched();
2316     if (test_thread_flag(TIF_NEED_RESCHED))
2317         goto need_resched;
2318 }

```

第 2288 行：试图把进程 `next` 的 `task` 结构的内容预取到 CPU 的 L1 高速缓存（详情可参阅 `include/linux/prefetch.h`）。

第 2290 行：必须通知当前 CPU 目前正在进行上下文切换。这样，具有多 CPU 的设备才能够确保正在被其他 CPU 共享的数据可被独占地访问。这个过程称为 RCU。详情可参阅 <http://lse.sourceforge.net/locking/rcupdate.html>。

第 2292~2298 行：将进程 `prev` 的 `sleep_avg` 属性减去它已经运行的时间，并对负值进行调整。如果该进程既不是交互式的也不是非交互式的，其交互信用就处于低信用和高信用之间，由于进程 `prev` 的平均睡眠时间很短，因此，降低其交互信用，并将其时间戳更新为当前时间。这样可以帮助调度程序了解给定进程已经使用了多少 CPU 时间，继而估算进程以后将要使用多少 CPU 时间。

第 2300~2304 行：如果调度程序没有选择同一个进程，就设置新进程的时间戳，增加运行队列的计数器，并把新进程设置为当前进程。

第 2306~2308 行：这几行代码描述了用汇编语言编写的 `context_switch()`。下一节深入讨论上下文切换时，还会谈到这个函数。

第 2314~2318 行：重新获得内核锁，启用抢占，并判断是否需要立刻重新调度；如果需要重新调度，则回到 `schedule()` 的开头。

执行完 `context_switch()` 后，有可能需要重新调度。因为，也许 `scheduler_tick()` 已经把一个新进程标记为需要重新调度了，或者还有一种情况：当启用抢占时，新进程已经被标记为需要重新调度了。我们继续重新调度进程（并对它们进行上下文切换），直到找到一个不需要重新调度的进程为止。离开 `schedule()` 的进程成为将要在这个 CPU 上执行的新进程。

7.1.2 上下文切换

`/kernel/sched.c` 中的 `schedule()` 调用了 `context_switch()`，该函数用来完成与硬件相关的内存环境和处理器状态的切换。一言以蔽之，上下文切换就是指用下一个进程来交换当前进程。函数 `context_switch()` 执行下一个进程并返回指向前一个进程的进程结构的指针。

```

-----
kernel/sched.c
1048 /*
1049 * context_switch - switch to the new MM and the new
1050 * thread's register state.
1051 */
1052 static inline
1053 task_t * context_switch(runqueue_t *rq, task_t *prev, task_t *next)
1054 {
1055     struct mm_struct *mm = next->mm;
1056     struct mm_struct *oldmm = prev->active_mm;
1057     ...
1063     switch_mm(oldmm, mm, next);
1058     ...
1072     switch_to(prev, next, prev);
1073
1074     return prev;
1075 }
-----

```

此处，描述了 context_switch 的两个工作：一个是切换虚拟内存映射；另一个是切换进程/线程的结构。第一个工作是由函数 switch_mm() 完成的，该函数使用了许多与硬件相关的内存管理的结构和寄存器：

```

-----
#include/asm-i386/mmu_context.h
026 static inline void switch_mm(struct mm_struct *prev,
027     struct mm_struct *next,
028     struct task_struct *tsk)
029 {
030     int cpu = smp_processor_id();
031
032     if (likely(prev != next)) {
033         /* stop flush ipis for the previous mm */
034         cpu_clear(cpu, prev->cpu_vm_mask);
035 #ifdef CONFIG_SMP
036         cpu_tlbstate[cpu].state = TLBSTATE_OK;
037         cpu_tlbstate[cpu].active_mm = next;
038 #endif
039         cpu_set(cpu, next->cpu_vm_mask);
040
041         /* Re-load page tables */
042         load_cr3(next->pgd);
043
044         /*
045          * load the LDT, if the LDT is different:
046          */
047         if (unlikely(prev->context.ldt != next->context.ldt))
048             load_LDT_nolock(&next->context, cpu);
049     }
050 #ifdef CONFIG_SMP
051     else {
052         ...
053     }
054 }
-----

```

第 39 行：将新进程绑定到当前处理器。

第 42 行：用来切换内存上下文的代码使用了 x86 的硬件寄存器 cr3，该寄存器中存放了为给定进程进行所有分页操作的基地址。新的页全局描述符即 next->pgd 被加载到 cr3。

第 47 行：大多数进程共享同一个 LDT。如果该进程请求另一个 LDT，则从新进程的 next->context 结构中把这个 LDT 加载到此处。

接着，/kernel/sched.c 中的 context_switch() 的另一半代码调用了宏 switch_to()，这个宏再调用 C 函数 __switch_to()。对于 x86 和 PPC 而言，体系结构独立的部分和体系结构相关的部分，其分界线都是宏 switch_to()。

1. 追踪 x86 中 switch_to() 的踪迹

x86 的代码比 PPC 的代码更为紧凑。下面是为 __switch_to() 编写的与体系结构相关的代码。task_struct（不是 thread_struct）被传递到 __switch_to()。接下来要讨论的代码就是为调用 C 函数 __switch_to()（第 23 行）而设计的内联汇编代码，__switch_to() 以适当的 task_struct 结构作为参数。

context_switch 用于获得三个进程的指针：prev、next 和 last。此外，还有当前进程的指针。

从宏观的角度来看，调用 switch_to() 时将会发生什么？调用 switch_to() 后进程的指针发生了什么变化？

图 7-2 表示分别用进程 A、B、C 对 switch_to() 进行调用。

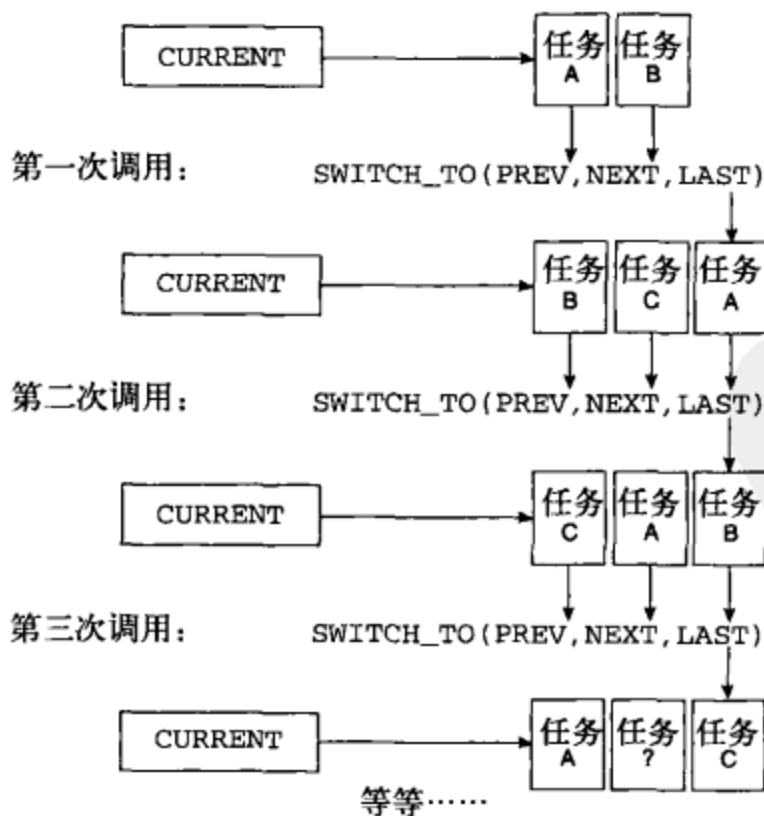


图 7-2 switch_to 调用

我们想切换进程 A 和进程 B。在第一次调用 switch_to() 之前：

- current → A
- prev → A, next → B

第一次调用 `switch_to()` 之后:

□ `current` → B

□ `last` → A

现在, 我们想切换进程 B 和进程 C。在第二次调用 `switch_to()` 之前:

□ `current` → B

□ `prev` → B, `next` → C

第二次调用 `switch_to()` 之后:

□ `current` → C

□ `last` → B

从第二个调用返回后, `current` 指向了进程 C, `last` 指向了进程 B。

进程 A 再一次被切换进来, 这个过程将周而复始地执行下去……

`switch_to()` 函数中的内联汇编是内核中汇编技巧应用的精彩实例, 也是 gcc C 扩展的一个好例子。第 2 章专门描述过这个函数的特色。接下来仔细分析这段代码。

```
-----
#include/asm-i386/system.h
012 extern struct task_struct * FASTCALL(__switch_to(struct task_struct *prev,
struct task_struct *next));

015 #define switch_to(prev,next,last) do {      \
016     unsigned long esi,edi;                  \
017     asm volatile("pushfl\n\t"              \
018     "pushl %%ebp\n\t"                        \
019     "movl %%esp,%0\n\t" /* save ESP */      \
020     "movl %5,%%esp\n\t" /* restore ESP */   \
021     "movl $1f,%1\n\t" /* save EIP */        \
022     "pushl %6\n\t" /* restore EIP */       \
023     "jmp __switch_to\n\t"                  \
023     "1:\n\t"                                \
024     "popl %%ebp\n\t"                        \
025     "popfl"                                  \
026     : "=m" (prev->thread.esp), "=m" (prev->thread.eip), \
027     "=a" (last), "=S" (esi), "=D" (edi)    \
028     : "m" (next->thread.esp), "m" (next->thread.eip), \
029     "2" (prev), "d" (next));                \
030 } while (0)
-----
```

第 12 行: `FASTCALL` 宏解析 `__attribute__((regparm(3)))`, 它强制将参数传入寄存器, 而不是栈。

第 15~16 行: `do {} while (0)` 结构使得这个宏拥有两个局部变量 `esi` 和 `edi`。注意: `esi` 和 `edi` 只是用熟悉的名字命名的局部变量。

current 和 task 结构

分析内核时, 无论何时需要获取或者存储正在给定处理器上运行的任务 (或进程) 的信息, 都必须利用全局变量 `current` 来引用其进程结构。例如, `current->pid` 包含进程的 ID。Linux

可以用一个既快速又巧妙的方法来引用当前进程的进程结构。

进程创建时，每个进程都被分配 8 K 的连续内存。(Linux 2.6 有一个编译时选项，通过这个选项可以使用 4 K 内存，而不是 8 K) 这个 8 K 的段由进程结构和为给定进程所设置的内核栈占用。当进程创建时，Linux 把进程结构放置在 8 K 内存的低端，而内核栈的指针则从高端开始。当数据被压到栈中时，内核栈的指针（具体是对 x86 和 PPC 的 `r1` 来说）随着数据的压入而减小。由于 8 K 的内存区是页对齐的，所以，该内存区的起始地址（用十六进制标识）总是以 0x000 结束（4 k 字节的倍数）。

读者可能已经猜到了，Linux 用来引用当前 task 结构的巧妙方法是用 `0xffff_f000` 和栈指针的内容进行“与”操作。通过让通用寄存器 2 保存当前进程的指针，PPC Linux 内核的最新版本把这个操作向前推进了一步。

第 17 行和第 30 行：`asm volatile()`^①这种形式封装了一个内联汇编代码块，`volatile` 关键字确保编译器不会以任何方式改变（优化）例程。

第 17~18 行：把寄存器 `flags` 和 `ebp` 压入栈中。（注意，此时仍然在使用和进程 `prev` 相关的栈）。

第 19 行：把当前进程的栈指针 `esp` 保存到 `prev` 的 task 结构。

第 20 行：把栈指针从 `next` 的 task 结构移动至当前处理器的 `esp`。

注意 迄今为止，还只是在定义上进行了上下文切换。

现在，我们有了一个新的内核栈，因而，任何对 `current` 的引用均被指向新的（即 `next`）的 task 结构。

第 21 行：将 `prev` 进程的返回地址保存到你 task 结构。当 `prev` 进程重新启动时，就从该地址恢复执行。

第 22 行：把返回地址（从 `__switch_to()` 返回的地址）压入栈中，这是来自于 `next` 的 `eip`。当进程被停止或者最近一次被抢占时，`eip` 将被保存到该进程的 task 结构中（在第 21 行）。

第 23 行：跳转到 C 函数 `__switch_to()`，更新下列信息。

- ☐ 下一个含有内核栈指针的线程结构。
- ☐ 当前处理器的线程局部存储器描述符。
- ☐ `prev` 和 `next` 的 `fs` 及 `gs`（如果需要的话）。
- ☐ 调试寄存器（如果需要的话）。
- ☐ I/O 位图（如果需要的话）。

然后，`__switch_to()` 返回更新之后的 `prev` 的进程结构。

第 24~25 行：从新的即 `next` 进程的内核栈弹出基址指针和标志寄存器。

第 26~29 行：这是传给内联汇编例程的输入参数和输出参数。关于对这些参数所施加的限制，

① 有关 `volatile` 的更多信息请参阅第 2 章。

详情请参阅 2.4 节。

第 29 行：依靠神奇的汇编代码，prev 被返回到 eax 中，它在顺序上是第三个参数。换句话说，输入参数 prev 作为输出参数 last 传给 switch_to() 宏。

因为 switch_to() 是一个宏，它内嵌在 context_switch() 中调用它的代码里执行。它不会像通常的函数那样返回。

为了清楚起见，注意，switch_to() 把 prev 传回 eax 寄存器，然后，在 context_switch() 中继续执行，其下一条指令是返回 prev (kernel/sched.c 的第 1074 行)。这使得 context_switch() 传回一个指向上一次运行的进程的指针。

2. 跟踪 PPC 的 context_switch()

要使 context_switch() 的 PPC 代码和 x86 有相同的效果，就必须稍微多做一些工作。和 x86 体系结构中的 cr3 寄存器不同，PPC 使用散列函数指向上下文环境。为 switch_mm() 编写的下列代码涉及了这些散列函数，第 4 章中详细解释了这些函数。

此处是 switch_mm() 例程，它依次调用 set_context() 例程。

```
-----
#include/asm-ppc/mmu_context.h
155 static inline void switch_mm(struct mm_struct *prev, struct
mm_struct *next, struct task_struct *tsk)
156 {
157     tsk->thread.pgdir = next->pgd;
158     get_mmu_context(next);
159     set_context(next->context, next->pgd);
160 }
-----
```

第 157 行：新线程的页全局目录（段寄存器）指向 next->pgd 指针。

第 158 行：传入 switch_mm() 的 mm_struct 的 context 字段 (next->context) 被更新为适当的上下文值。上下文的信息来自于对变量 context_map[] 的全局引用，context_map[] 含有一组位图字段。

第 159 行：此处是对汇编例程 set_context 的调用。下面是这个例程的代码以及对它的讨论，当第 1468 行的 blr 指令执行时，代码将返回到 switch_mm 例程。

```
-----
/arch/ppc/kernel/head.S
1437 _GLOBAL(set_context)
1438 mulli r3,r3,897 /* multiply context by skew factor */
1439 rlwinm r3,r3,4,8,27 /* VSID = (context & 0xffff) << 4 */
1440 addis r3,r3,0x6000 /* Set Ks, Ku bits */
1441 li r0,NUM_USER_SEGMENTS
1442 mtctr r0
...
1457 3: isync
...
1461 mtsrin r3,r4
1462 addi r3,r3,0x111 /* next VSID */
1463 rlwinm r3,r3,0,8,3 /* clear out any overflow from VSID field */
-----
```

```

1464 addis r4,r4,0x1000 /* address of next segment */
1465 bdnz 3b
1466 sync
1467 isync
1468 blr
-----

```

第 1437~1440 行：经由 r3 传递给 set_context() 的 mm_struct 的 context 字段 (next->context) 为 PPC 的分段安装散列函数。

第 1461~1465 行：经由 r4 传递给 set_context() 的 mm_struct 的 pgd 字段 (next->pgd) 指向段寄存器。

分段是 PPC 内存管理的基础 (参阅第 4 章)。当从 set_context() 返回时，将把 mm_struct 类型的 next 初始化成合适的内存区域并返回到 switch_mm()。

3. 跟踪 switch_to() 的 PPC 踪迹

PPC 对 switch_to() 的实现必然和 x86 的 switch_to() 调用完全相同；它接受 current 和 next 的进程指针，并返回指向前一个运行进程的指针。

```

-----
include/asm-ppc/system.h
88 extern struct task_struct *__switch_to(struct task_struct *,
89   struct task_struct *);
90 #define switch_to(prev, next, last)
91 ((last) = __switch_to((prev), (next)))
92
91
92 struct thread_struct;
93 extern struct task_struct *_switch(struct thread_struct *prev,
94   struct thread_struct *next);
-----

```

在第 88 行，__switch_to() 将其参数视为 task_struct 类型的，而在第 93 行，_switch() 将其参数视为 thread_struct 类型的。这是因为，task_struct 中的线程入口含有特定线程所关心的、与体系结构相关的处理器寄存器信息。现在，先来分析 __switch_to() 的实现。

```

-----
/arch/ppc/kernel/process.c
200 struct task_struct *__switch_to(struct task_struct *prev,
   struct task_struct *new)
201 {
202   struct thread_struct *new_thread, *old_thread;
203   unsigned long s;
204   struct task_struct *last;
205   local_irq_save(s);
   ...
247   new_thread = &new->thread;
248   old_thread = &current->thread;
249   last = _switch(old_thread, new_thread);
250   local_irq_restore(s);
251   return last;
252 }
-----

```

第 205 行：上下文切换前禁止中断。

第 247~248 行：仍然运行在老线程的上下文中，把指向线程结构的指针传给 `_switch()` 函数。

第 249 行：`_switch()` 是为完成两个线程结构的切换而调用的汇编程序（参见下一节）。

第 250 行：上下文切换后开启中断。

为了更好地理解 PPC 线程内部的哪些信息必须被交换，必须考查在第 249 行传入的 `thread_struct`。

回想一下对 x86 上下文切换所做的研究，直到指向一个新的内核栈时，上下文切换才会正式生效。在 PPC 中，让上下文切换正式生效的这关键一步正是发生在 `_switch()` 中。

● 追踪 PPC 的 `_switch()` 代码

按照惯例，PPC 中 C 函数的参数（从左至右）分别保存在 `r3`、`r4`、`r5...r12` 中。进入 `switch()` 时，`r3` 指向当前进程的 `thread_struct`，`r4` 指向新进程的 `thread_struct`。

```
-----
/arch/ppc/kernel/entry.S
437 _GLOBAL(_switch)
438 stwu r1,-INT_FRAME_SIZE(r1)
439 mflr r0
440 stw r0,INT_FRAME_SIZE+4(r1)
441 /* r3-r12 are caller saved -- Cort */
442 SAVE_NVGPRS(r1)
443 stw r0,_NIP(r1) /* Return to switch caller */
444 mfmsr r11
...
458 1: stw r11,_MSR(r1)
459 mfcrr r10
460 stw r10,_CCR(r1)
461 stw r1,KSP(r3) /* Set old stack pointer */
462
463 tophys(r0,r4)
464 CLR_TOP32(r0)
465 mtspr SPRG3,r0/* Update current THREAD phys addr */
466 lwz r1,KSP(r4) /* Load new stack pointer */
467 /* save the old current 'last' for return value */
468 mr r3,r2
469 addi r2,r4,-THREAD /* Update current */
...
478 lwz r0,_CCR(r1)
479 mtcrrf 0xFF,r0
480 REST_NVGPRS(r1)
481
482 lwz r4,_NIP(r1) /* Return to _switch caller in new task */
483 mtlr r4
484 addi r1,r1,INT_FRAME_SIZE
485 blr
-----
```

逐字节的为新线程换出前一个 `thread_struct` 的机制作为练习留给大家。然而，值得注意的是，`_switch()` 中对 `r1`、`r2`、`r3`、`SPRG3` 和 `r4` 的使用参见这种操作的基本知识。

第 438~460 行：上下文环境被保存到和当前栈指针 r1 相关的当前栈中。

第 461 行：整个环境被存入经由 r3 传入的当前 thread_struct 的指针。

第 463~465 行：SPRG3 被更新为指向新进程的线程结构。

第 466 行：KSP 是 task 结构 (r4) 的偏移，新进程的内核栈指针指向这个 task 结构。现在，栈指针 r1 可以用这个值 KSP 来更新。(这一步是 PPC 上下文切换的要点。)

第 468 行：指向前一个进程的当前指针从 _switch() 返回后放在 r3 中。这表示上一个进程。

第 469 行：用指向新进程结构的指针 (r4) 来更新当前指针 (r2)。

第 478~486 行：从新栈中恢复环境的剩余部分，把 r3 中的前一个进程的结构返回给调用程序。

对 context_switch() 的分析到这里就结束了。下面的代码表示：随着被 schedule() 中的 context_switch 调用，处理器交换 prev 和 next 这两个进程。

```
-----
kernel/sched.c
1709 prev = context_switch(rq, prev, next);
-----
```

现在，prev 指向了刚才切换掉的进程，next 则指向了当前进程。

既然已经讨论了在 Linux 内核中如何调度进程，接下来就考查如何告知进程它将要被调度，即什么因素导致了 schedule() 的调用，什么因素又导致了一个进程不得不为另一个进程让出 CPU？

7.1.3 让出 CPU

只需调用 schedule() 就可使进程自动放弃 CPU。在想要睡眠或等待信号发生的内核代码以及设备驱动程序中，这个函数用得非常普遍^①。若其他进程想要连续地使用 CPU，系统定时器就必须告诉它们让出 CPU。Linux 内核定期地夺取 CPU，以此来强制停止活跃的进程，然后执行许多基于定时器的任务。scheduler_tick() 就是其中之一，该内核函数强迫一个进程放弃 CPU。如果一个进程运行时间过长，内核就不会把 CPU 的控制权交还给这个进程，而是改为选择另外一个进程。现在，给读者介绍 scheduler_tick() 如何判断当前进程是否必须放弃 CPU：

```
-----
kernel/sched.c
1981 void scheduler_tick(int user_ticks, int sys_ticks)
1982 {
1983     int cpu = smp_processor_id();
1984     struct cpu_usage_stat *cpustat = &kstat_this_cpu.cpustat;
1985     runqueue_t *rq = this_rq();
1986     task_t *p = current;
1987
1988     rq->timestamp_last_tick = sched_clock();
1989
-----
```

^① Linux 规范指出：在持有自旋锁的时候不能调用调度程序，因为这可能引起系统死锁。这是忠告！

```

1990  if (rcu_pending(cpu))
1991      rcu_check_callbacks(cpu, user_ticks);
-----

```

第 1981~1986 行：这个代码块初始化 `scheduler_tick()` 函数所必需的数据结构。`cpu`、`cpu_usage_stat` 和 `rq` 分别被设置为处理器 ID、CPU 状态和当前处理器的运行队列。`p` 是一个指针，指向正在使用 CPU 的当前进程。

第 1988 行：运行队列的最后一个节拍被设置成以纳秒为单位的当前时间。

第 1990~1991 行：在 SMP 系统中，必须检查是否有未完成的 RCU 需要执行，如果有的话就通过 `rcu_check_callbacks()` 来执行。

```

-----
kernel/sched.c
1993  /* note: this timer irq context must be accounted for as well */
1994  if (hardirq_count() - HARDIRQ_OFFSET) {
1995      cpustat->irq += sys_ticks;
1996      sys_ticks = 0;
1997  } else if (softirq_count()) {
1998      cpustat->softirq += sys_ticks;
1999      sys_ticks = 0;
2000  }
2001
2002  if (p == rq->idle) {
2003      if (atomic_read(&rq->nr_iowait) > 0)
2004          cpustat->iowait += sys_ticks;
2005      else
2006          cpustat->idle += sys_ticks;
2007      if (wake_priority_sleeper(rq))
2008          goto out;
2009      rebalance_tick(cpu, rq, IDLE);
2010      return;
2011  }
2012  if (TASK_NICE(p) > 0)
2013      cpustat->nice += user_ticks;
2014  else
2015      cpustat->user += user_ticks;
2016  cpustat->system += sys_ticks;
-----

```

第 1994~2000 行：`cpustat` 跟踪内核的统计信息，并根据已经发生的系统节拍数来更新硬件中断和软件中断的统计信息。

第 2002~2011 行：如果当前没有进程运行，则认真地检查是否有进程正在等待 I/O。如果有就增加 CPU 的 I/O 等待统计计数；否则，增加 CPU 的空闲统计计数。在单处理器系统中，`rebalance_tick()` 毫无意义，但在多处理器系统中，由于这个 CPU 没有事情可做，`rebalance_tick()` 就会尽力对当前的 CPU 进行负载平衡。

第 2012~2016 行：这个代码块完成更多 CPU 统计信息的收集。如果当前进程被礼让，就增加 CPU 的 `nice` 计数器；否则，增加用户的节拍计数器。最后，增加 CPU 的系统节拍计数器。


```

-----
kernel/sched.c
2019  if (p->array != rq->active) {
2020      set_tsk_need_resched(p);
2021      goto out;
2022  }
2023  spin_lock(&rq->lock);
-----

```

第 2019~2022 行：此处指明为什么要把指向优先级数组的指针存入进程的 `task_struct`。调度程序检查当前进程是否不再活跃。如果进程已经到期，调度程序将设置进程的重新调度标志，并跳转到 `scheduler_tick()` 函数的末尾。在那里（第 2092~2093 行），由于还没有活跃的进程，调度程序就尝试对这个 CPU 进行负载平衡。在当前进程能够调度自己或从一个成功的运行返回之前，调度程序在夺取 CPU 控制权时就会发生这种情况。

第 2023 行：得知当前进程正在运行，它没有到期并一直存在。现在，调度程序想给另外的进程让出 CPU 的控制权；它必须做的第一件事情就是获得运行队列锁。

```

-----
kernel/sched.c
2024  /*
2025  * The task was running during this tick - update the
2026  * time slice counter. Note: we do not update a thread's
2027  * priority until it either goes to sleep or uses up its
2028  * timeslice. This makes it possible for interactive tasks
2029  * to use up their timeslices at their highest priority levels.
2030  */
2031  if (unlikely(rt_task(p))) {
2032      /*
2033       * RR tasks need a special form of timeslice management.
2034       * FIFO tasks have no timeslices.
2035       */
2036      if ((p->policy == SCHED_RR) && !--p->time_slice) {
2037          p->time_slice = task_timeslice(p);
2038          p->first_time_slice = 0;
2039          set_tsk_need_resched(p);
2040
2041          /* put it at the end of the queue: */
2042          dequeue_task(p, rq->active);
2043          enqueue_task(p, rq->active);
2044      }
2045      goto out_unlock;
2046  }
-----

```

第 2031~2046 行：对于调度程序来说，最简单的情形莫过于当前进程是实时进程。实时进程总是拥有比其他任何进程都高的优先级。如果该进程是 FIFO 进程并且正在运行，就应该继续它的操作，因此，直接跳转到函数的末尾并释放运行队列锁；如果当前进程是时间片轮转（round-robin）的实时进程，就减小其时间片；如果进程没有更多的时间片，就应该调度另一个

轮转实时进程。通过 `task_timeslice()` 为当前进程计算新的时间片。接着，重新设置该进程的第一个时间片。然后，进程被标记为需要重新调度，最后，从运行队列的活跃数组删除该进程并把它添加回活跃数组，该进程被放到轮转实时进程链表的末尾。此时，调度程序跳转到函数的末尾并释放运行队列锁。

```
-----
kernel/sched.c
2047  if (!--p->time_slice) {
2048      dequeue_task(p, rq->active);
2049      set_tsk_need_resched(p);
2050      p->prio = effective_prio(p);
2051      p->time_slice = task_timeslice(p);
2052      p->first_time_slice = 0;
2053
2054      if (!rq->expired_timestamp)
2055          rq->expired_timestamp = jiffies;
2056      if (!TASK_INTERACTIVE(p) || EXPIRED_STARVING(rq)) {
2057          enqueue_task(p, rq->expired);
2058          if (p->static_prio < rq->best_expired_prio)
2059              rq->best_expired_prio = p->static_prio;
2060      } else
2061          enqueue_task(p, rq->active);
2062  } else {
-----
```

第 2047~2061 行：此处，调度程序知道当前进程不是实时进程。它减小进程的时间片，在这里，进程的时间片终会被耗尽到 0。调度程序从活跃数组中删除进程并设置它的需要重新调度标志。然后重新计算进程的优先级，并重新设置它的时间片。优先级的计算和时间片的重置都需要考虑进程之前的活动^①。如果运行队列的到期时间戳为 0（通常发生在运行队列的活跃数组上没有更多进程的时候），就把运行队列的到期时间戳设置为 `jiffies`。

jiffies

`jiffies` 是 32 位的变量，计算系统自引导以来发生的节拍数。在一个 100Hz 的系统中，这个数回绕到 0 大约得花 497 天。第 20 行的宏是以 `u64` 类型访问这个值的建议方法。在 `include/jiffies.h` 中也有一些用于检测的宏。

```
-----
include/linux/jiffies.h
017  extern unsigned long volatile jiffies;
020  u64 get_jiffies_64(void);
-----
```

通常，通过把交互式进程重新放回运行队列的活跃优先级数组来给予它们一些照顾，这便是第 2060 行的 `else` 子句。然而，也不应该为此饿死到期进程。我们使用 `EXPIRED_STARVING()`

^① 参见 `effective_prio()` 和 `task_timeslice()`。

(参见第 1968 行的 EXPIRED_STARVING) 来判断到期进程等待 CPU 的时间是否过长。如果第一个到期进程正在等待不合理的时间量, 或者如果到期数组含有优先级高于当前进程的进程, 函数 EXPIRED_STARVING() 就返回真。等待的时间数合理与否与负载有关, 运行的进程数目越多, 活跃数组和到期数组的交换次数就越少。

如果进程不是交互式的, 或者到期进程正处于饥饿状态, 调度程序将获取当前进程并把它放入运行队列的到期优先级数组中。如果当前进程的静态优先级高于到期运行队列中优先级最高的进程的优先级, 就更新运行队列以反映到期数组现在的优先级比之前的优先级更高。(注意, 在 Linux 中, 进程的优先级越高, 编号越小, 因此, 代码中用了 <。)

```
-----
kernel/sched.c
2062  } else {
2063      /*
2064       * Prevent a too long timeslice allowing a task to monopolize
2065       * the CPU. We do this by splitting up the timeslice into
2066       * smaller pieces.
2067       *
2068       * Note: this does not mean the task's timeslices expire or
2069       * get lost in any way, they just might be preempted by
2070       * another task of equal priority. (one with higher
2071       * priority would have preempted this task already.) We
2072       * requeue this task to the end of the list on this priority
2073       * level, which is in essence a round-robin of tasks with
2074       * equal priority.
2075       *
2076       * This only applies to tasks in the interactive
2077       * delta range with at least TIMESLICE_GRANULARITY to requeue.
2078       */
2079      if (TASK_INTERACTIVE(p) && !((task_timeslice(p) -
2080          p->time_slice) % TIMESLICE_GRANULARITY(p)) &&
2081          (p->time_slice >= TIMESLICE_GRANULARITY(p)) &&
2082          (p->array == rq->active)) {
2083
2084          dequeue_task(p, rq->active);
2085          set_tsk_need_resched(p);
2086          p->prio = effective_prio(p);
2087          enqueue_task(p, rq->active);
2088      }
2089  }
2090 out_unlock:
2091  spin_unlock(&rq->lock);
2092 out:
2093  rebalance_tick(cpu, rq, NOT_IDLE);
2094 }
-----
```

第 2079~2089 行: 调度程序执行之前的最后一种情形是当前进程正在运行并且还有剩余的时间片可运行。调度程序必须确保拥有大块时间片的进程不会独占 CPU。如果进程是交互式的, 拥

有大于 `TIMESLICE_GRANULARITY` 的时间片且是活跃的，那么调度程序将从活跃队列删除它。于是，进程让它的重新调度标志置位，在重新计算其优先级后被放回运行队列的活跃数组。这样做的目的是确保存在拥有大块时间片的具有某一优先级的进程时，不会饿死同一优先级的其他进程。

第 2092~2094 行：调度程序完成运行队列的重新排列并对它解锁；如果是在 SMP 系统中执行，那么还要进行负载平衡。

如何通过 `scheduler_tick()` 把进程标记为重新调度，以及如何通过 `schedule()` 来调度进程，这两者结合起来说明了在 Linux 2.6 内核中调度程序是如何运转的。接下来将深入研究“优先级”对调度程序的意义。

1. 动态优先级的计算

前一节没有介绍如何计算进程动态优先级的细节。进程的优先级基于它先前的行为以及用户指定的 `nice` 值。函数 `recalc_task_prio()` 用来为进程确定新的动态优先级：

```
-----
kernel/sched.c
381 static void recalc_task_prio(task_t *p, unsigned long long now)
382 {
383     unsigned long long __sleep_time = now - p->timestamp;
384     unsigned long sleep_time;
385
386     if (__sleep_time > NS_MAX_SLEEP_AVG)
387         sleep_time = NS_MAX_SLEEP_AVG;
388     else
389         sleep_time = (unsigned long)__sleep_time;
390
391     if (likely(sleep_time > 0)) {
392         /*
393          * User tasks that sleep a long time are categorised as
394          * idle and will get just interactive status to stay active &
395          * prevent them suddenly becoming cpu hogs and starving
396          * other processes.
397          */
398         if (p->mm && p->activated != -1 &&
399             sleep_time > INTERACTIVE_SLEEP(p)) {
400             p->sleep_avg = JIFFIES_TO_NS(MAX_SLEEP_AVG -
401                 AVG_TIMESLICE);
402             if (!HIGH_CREDIT(p))
403                 p->interactive_credit++;
404         } else {
405             /*
406              * The lower the sleep avg a task has the more
407              * rapidly it will rise with sleep time.
408              */
409             sleep_time *= (MAX_BONUS - CURRENT_BONUS(p)) ? : 1;
410
411             /*
412              * Tasks with low interactive_credit are limited to
413              * one timeslice worth of sleep avg bonus.
```

```

414     */
415     if (LOW_CREDIT(p) &&
416         sleep_time > JIFFIES_TO_NS(task_timeslice(p)))
417         sleep_time = JIFFIES_TO_NS(task_timeslice(p));
418
419     /*
420     * Non high_credit tasks waking from uninterruptible
421     * sleep are limited in their sleep_avg rise as they
422     * are likely to be cpu hogs waiting on I/O
423     */
424     if (p->activated == -1 && !HIGH_CREDIT(p) && p->mm) {
425         if (p->sleep_avg >= INTERACTIVE_SLEEP(p))
426             sleep_time = 0;
427         else if (p->sleep_avg + sleep_time >=
428                 INTERACTIVE_SLEEP(p)) {
429             p->sleep_avg = INTERACTIVE_SLEEP(p);
430             sleep_time = 0;
431         }
432     }
433
434     /*
435     * This code gives a bonus to interactive tasks.
436     *
437     * The boost works by updating the 'average sleep time'
438     * value here, based on ->timestamp. The more time a
439     * task spends sleeping, the higher the average gets -
440     * and the higher the priority boost gets as well.
441     */
442     p->sleep_avg += sleep_time;
443
444     if (p->sleep_avg > NS_MAX_SLEEP_AVG) {
445         p->sleep_avg = NS_MAX_SLEEP_AVG;
446         if (!HIGH_CREDIT(p))
447             p->interactive_credit++;
448     }
449 }
450 }
451
452 p->prio = effective_prio(p);
453 }

```

第 386~389 行：基于时间 `now` 来计算进程 `p` 已经睡眠了多长时间，并把计算得到的值赋给 `sleep_time`，`sleep_time` 的最大值是 `NS_MAX_SLEEP_AVG` (`NS_MAX_SLEEP_AVG` 默认为 10 毫秒)。

第 391~404 行：如果进程 `p` 已经睡眠了，那么首先检查它是否睡眠了足够长的时间（这个时间应足以把进程归类为交互式进程）。如果睡眠时间足够长，当 `sleep_time > INTERACTIVE_SLEEP(p)` 时，就把进程的平均睡眠时间调整为一个固定值，如果 `p` 还不能归类为交互式进程，就增加 `p` 的 `interactive_credit`。

第 405~410 行：给予平均睡眠时间短的进程更多的睡眠时间。

第 411~418 行：如果进程是 CPU 消耗型（CPU 密集型）的，它就被归类为非交互式的，那么就限定进程的时间片至多相当于平均睡眠时间的奖励值。

第 419~432 行：还没有归类为交互式的（非 HIGH_CREDIT 的），从不可中断的睡眠状态醒来的进程，将其平均睡眠时间限定为 INTERACTIVE()。

第 434~450 行：将重新计算过的 sleep_time 加到进程的平均睡眠时间上，要确保它不会超过 NS_MAX_SLEEP_AVG。如果进程被认为不是交互式的，但却睡眠了最大的时间甚至更长的时间，就增加其交互信用。

第 452 行：最后，根据刚刚计算出来的进程 p 的 sleep_avg 字段，利用 effective_prio() 来设置 p 的优先级，该函数通过把 0..MAX_SLEEP_AVG 的平均睡眠时间换算成 -5~+5 的数来完成优先级的设置。因此，依赖于其先前的行为，一个静态优先级为 70 的进程能够拥有一个 65~85 的动态优先级。

非实时进程的进程优先级在 101~140 的范围内变化。即使以非常高的优先级运行的进程（105 甚至更小一点），也不能超越实时进程的界限。显然，高优先级、高交互信用的进程决不可能拥有一个低于 101 的动态优先级（在默认配置中实时进程的优先级涵盖范围为 0~100）。

2. 从运行队列删除进程

前面已经讨论了如何通过分叉把进程插入调度程序，以及如何将进程从 CPU 运行队列中的活跃优先级数组移动至到期优先级数组。然而，究竟如何从运行队列删除进程呢？

主要有两种方法可以从运行队列中删除进程。

❑ 进程被内核抢占且它并不处于运行状态，并且进程没有挂起的信号（参见 kernel/sched.c 中的第 2240 行）。

❑ 在 SMP 机器中，进程能够从一个运行队列移出放到另一个运行队列中（参见 kernel/sched.c 中的第 3384 行）。

第一种情形通常发生在进程把自己放到等待队列上睡眠之后，schedule() 被调用之时。进程把自己标记为非运行状态（TASK_INTERRUPTIBLE、TASK_UNINTERRUPTIBLE、TASK_STOPPED 等），并且从运行队列彻底删除它以后，内核不再考虑该进程对 CPU 的访问。

进程被移动到另一个运行队列的情形在 Linux 内核的 SMP 部分处理，此处不作研究。

现在，我们来追踪如何通过 deactivate_task() 从运行队列删除进程：

```
-----
kernel/sched.c
507 static void deactivate_task(struct task_struct *p, runqueue_t *rq)
508 {
509     rq->nr_running--;
510     if (p->state == TASK_UNINTERRUPTIBLE)
511         rq->nr_uninterruptible++;
512     dequeue_task(p, p->array);
513     p->array = NULL;
514 }
-----
```

第 509 行：由于 p 不再运行，调度程序首先减小运行队列的运行进程数。

第 510~511 行：如果进程是不可中断的，就增加运行队列上不可中断进程的计数。相应地，当不可中断的进程醒来时则减小该计数。（参见 kernel/sched.c 的 `try_to_wake_up()` 函数中的第 824 行）。

第 512~513 行：现在，运行队列的统计信息全部更新了，接下来，将正式从运行队列删除进程 `p`。内核利用 `p->array` 字段检测进程是否正在运行以及是否在运行队列中。由于 `p->array` 不再属于上述这两种情况，因此把它设置为 `NULL`。

还有一些运行队列的管理工作需要完成；让我们来考查一下 `dequeue_task()` 的细节：

```
-----
kernel/sched.c
303 static void dequeue_task(struct task_struct *p, prio_array_t *array)
304 {
305     array->nr_active--;
306     list_del(&p->run_list);
307     if (list_empty(array->queue + p->prio))
308         __clear_bit(p->prio, array->bitmap);
309 }
-----
```

第 305 行：调整优先级数组中的活跃进程数，进程 `p` 不在到期数组上，就在活跃数组上。

第 306~308 行：根据 `p` 的优先级，从优先级数组中的相应进程链表中删除进程 `p`。如果删除操作导致链表为空，就必须清除优先级数组位图中的相应位，以表示再也没有任何进程的优先级为 `p->prio()`。

由于 `p->run_list` 是一个 `list_head` 结构，并有两个分别指向链表中前一个元素和下一个元素的指针，因此，仅用 `list_del()` 便可完成所有的删除操作。

至此，进程 `p` 从运行队列中消失了，完全不再是活跃进程，如果该进程处于 `TASK_NTERRUPTIBLE` 或 `TASK_UNINTERRUPTIBLE` 状态的，它还有被唤醒并回到运行队列的可能。如果进程处于 `TASK_STOPPED`、`TASK_ZOMBIE` 或 `TASK_DEAD` 状态的话，它的所有结构都要被删除和丢弃得干干净净。

7.2 内核抢占

内核抢占指从一个进程切换到另一个进程。前面已经讨论过 `schedule()` 和 `scheduler_tick()` 如何决定接下来要切换到哪一个进程，但仍然没有描述 Linux 内核如何确定进行切换的具体时机。Linux 2.6 内核引入了内核抢占机制，这意味着在任何时刻，不论是用户空间的程序还是内核空间的程序都能够被切换下来。因为内核抢占是 Linux 2.6 中的标准，下面将介绍 Linux 中内核抢占和用户抢占是如何进行的。

7.2.1 显式内核抢占

最容易理解的抢占是显式内核抢占。显式内核抢占发生在内核代码调用 `schedule()` 时的内核空间中。内核代码可通过两种方式调用 `schedule()`：直接调用 `schedule()`，或者自我阻塞。

当显式地抢占内核时，例如，设备驱动程序在等待队列 `wait_queue` 中等待时，控制权被简

单地交给调度程序，从而新的进程被选中执行。

7.2.2 隐式用户抢占

当内核处理完内核空间的进程，准备把控制权交给用户空间的进程时，它将首先查看应该把控制权交给用户空间的哪一个进程，这个进程也许不是之前将控制权交给内核的那个进程。例如，如果进程 A 调用了系统调用，那么该系统调用完成之后，内核可能把系统的控制权交给进程 B。

系统中的每个进程都有一个“需要重新调度”的标志，该标志在进程应该被重新调度的时候被设置。

```
-----
include/linux/sched.h
988 static inline void set_tsk_need_resched(struct task_struct *tsk)
989 {
990     set_tsk_thread_flag(tsk, TIF_NEED_RESCHED);
991 }
992
993 static inline void clear_tsk_need_resched(struct task_struct *tsk)
994 {
995     clear_tsk_thread_flag(tsk, TIF_NEED_RESCHED);
996 }
...
1003 static inline int need_resched(void)
1004 {
1005     return unlikely(test_thread_flag(TIF_NEED_RESCHED));
1006 }
-----
```

第 988~996 行：set_tsk_need_resched 和 clear_tsk_need_resched 是两个接口，用于设置和体系结构相关的标志 TIF_NEED_RESCHED。

第 1003~1006 行：need_resched 用于测试当前线程的标志 TIF_NEED_RESCHED 是否被设置。

如同 schedule() 和 scheduler_tick() 中描述的那样，当内核将要返回用户空间时，它将选择一个接受控制权的进程。尽管 scheduler_tick() 能够把进程标记为需要重新调度，但是这个标志只对 schedule() 有用。schedule() 反复选择一个新进程来执行，直到新选择的进程不需要重新调度为止。schedule() 完成后，新进程拥有处理器的控制权。

综上所述，当进程正在运行时，系统定时器引发一个触发 scheduler_tick() 的中断。scheduler_tick() 能够将那个进程标记为需要重新调度并把它移动到到期数组中。当内核操作完成时，scheduler_tick() 后面可能紧跟着其他中断，这样，内核将继续拥有对处理器的控制权，调用 schedule() 选择下一个要运行的进程。这样看来，scheduler_tick() 标记进程并重排运行队列，而 schedule() 选择下一个进程并传递 CPU 的控制权。

7.2.3 隐式内核抢占

Linux 2.6 中新增了对隐式内核抢占的实现。当内核进程拥有对 CPU 的控制权时，仅当其目前没有持有任何锁时，才能被另一个内核进程所抢占。每个进程有一个 preempt_count 字段，

用于标记进程是否可以被抢占。每当进程获得一次锁时，该计数增加；每当释放一次锁时，该计数减少。schedule()函数在决定接下来要运行哪个进程期间是禁止抢占的。

隐式内核抢占有两种可能性：内核代码出自禁止抢占的代码块，或者处理过程正在从中断返回内核代码。如果控制权正在从一个中断返回到内核空间，该中断调用 schedule()并以刚才描述的方式选中一个新进程。

如果内核代码出自禁止抢占的代码块，启用抢占的操作可能引起当前进程被抢占：

```
-----
include/linux/preempt.h
46 #define preempt_enable() \
47 do { \
48     preempt_enable_no_resched(); \
49     preempt_check_resched(); \
50 } while (0)
-----
```

第 46~50 行：preempt_enable()调用 preempt_enable_no_resched()，后者将当前进程的 preempt_count 减 1，然后调用 preempt_check_resched()：

```
-----
include/linux/preempt.h
40 #define preempt_check_resched() \
41 do { \
42     if (unlikely(test_thread_flag(TIF_NEED_RESCHED))) \
43         preempt_schedule(); \
44 } while (0)
-----
```

第 40~44 行：preempt_check_resched()判断当前进程是否被标记为重新调度：如果是，则调用 preempt_schedule()。

```
-----
kernel/sched.c
2328 asmlinkage void __sched preempt_schedule(void)
2329 {
2330     struct thread_info *ti = current_thread_info();
2331
2332     /*
2333      * If there is a non-zero preempt_count or interrupts are disabled,
2334      * we do not want to preempt the current task. Just return..
2335      */
2336     if (unlikely(ti->preempt_count || irqs_disabled()))
2337         return;
2338
2339     need_resched:
2340     ti->preempt_count = PREEMPT_ACTIVE;
2341     schedule();
2342     ti->preempt_count = 0;
2343
2344     /* we could miss a preemption opportunity between schedule and now */
2345     barrier();
-----
```

```

2346  if (unlikely(test_thread_flag(TIF_NEED_RESCHED)))
2347      goto need_resched;
2348  }

```

第 2336~2337 行：如果当前进程的 `preempt_count` 值仍然是正的（可能由于嵌套的 `preempt_disable()` 命令）或者当前进程禁用了中断，就把处理器的控制权交还给当前进程。

第 2340~2347 行：`preempt_count` 为 0 说明当前进程不持有任何锁并且允许 IRQ。因此，将当前进程的 `preempt_count` 设置成表明它正在经历抢占的状态，并调用 `schedule()` 来选择另一个进程。

如果进程出自需要重新调度的代码块，内核就必须确保当前进程放弃处理器是安全的。内核将检查进程的 `preempt_count` 值。如果 `preempt_count` 为 0，表示当前进程不持有锁，则 `schedule()` 被调用，一个新的进程被选中执行；如果 `preempt_count` 非 0，这时，把控制权传递给其他进程是危险的，于是，控制权被交还给当前进程，直到当前进程释放它的全部锁为止。当前进程释放锁时，要进行一个测试，判断其是否需要被重新调度。待当前进程释放它的最后一个锁后，`preempt_count` 就变成了 0，于是立刻进行调度。

7.3 自旋锁和信号量

当两个或多于两个进程请求独占地访问共享资源时，它们也许需要具备这样一种条件，即它们是在给定代码段中操作的唯一进程。Linux 内核中锁的基本形式是自旋锁。

自旋锁是因其连续地循环等待或连续地旋转等待以获得一个锁而得名。由于自旋锁的这种操作方式，禁止自旋锁中的任何代码段企图两次获得锁是非常必要的，否则将导致死锁。

对一个自旋锁进行操作前，必须初始化 `spin_lock_t`。这可通过调用 `spin_lock_init()` 来完成：

```

-----
include/linux/spinlock.h
63 #define spin_lock_init(x) \
64  do { \
65      (x)->magic = SPINLOCK_MAGIC; \
66      (x)->lock = 0; \
67      (x)->babble = 5; \
68      (x)->module = __FILE__; \
69      (x)->owner = NULL; \
70      (x)->oline = 0; \
71  } while (0)
-----

```

这段代码在第 66 行设置自旋锁为“开锁”状态（或 0）并初始化结构中的其他变量。此处，我们所关心的变量是 `(x)->lock`。

自旋锁被初始化后，可以通过调用 `spin_lock()` 或 `spin_lock_irqsave()` 来获取它。`spin_lock_irqsave()` 函数在上锁前禁止中断，而 `spin_lock()` 不会禁止中断。如果使用 `spin_lock()`，进程是有可能在上锁的代码段中被中断的。

为了在代码的临界区执行后释放锁，必须调用 `spin_unlock()` 或 `spin_unlock_irqrestore()`。`spin_unlock_irqrestore()` 把中断寄存器的状态恢复成调用 `spin_lock_irq()` 时的状态。

接下来考查一下 `spin_lock_irqsave()` 和 `spin_unlock_irqrestore()`：

```
-----
include/linux/spinlock.h
258 #define spin_lock_irqsave(lock, flags) \
259 do { \
260     local_irq_save(flags); \
261     preempt_disable(); \
262     _raw_spin_lock_flags(lock, flags); \
263 } while (0)
...
321 #define spin_unlock_irqrestore(lock, flags) \
322 do { \
323     _raw_spin_unlock(lock); \
324     local_irq_restore(flags); \
325     preempt_enable(); \
326 } while (0)
-----
```

注意，在上锁期间是如何禁止抢占的。禁止中断可确保临界区中的任何操作不被打断。第 324 行恢复了第 260 行保存的中断请求标志。

自旋锁的缺点是频繁地循环等待锁的释放。将它们用于可快速完成的代码临界区是最好不过的。对于花费时间较多的代码区，最好使用 Linux 内核的另一个上锁工具：信号量。

信号量与自旋锁的不同之处在于，当进程企图获得一个竞争资源时，进程是睡眠的而不是忙于等待的。信号量的主要优势之一是：持有信号量的进程可以安全地阻塞，它们在 SMP 和中断中都是安全的：

```
-----
include/asm-i386/semaphore.h
44 struct semaphore {
45     atomic_t count;
46     int sleepers;
47     wait_queue_head_t wait;
48 #ifdef WAITQUEUE_DEBUG
49     long __magic;
50 #endif
51 };
-----
include/asm-ppc/semaphore.h
24 struct semaphore {
25     /*
26     * Note that any negative value of count is equivalent to 0,
27     * but additionally indicates that some process(es) might be
28     * sleeping on 'wait'.
29     */
30     atomic_t count;
```

```

31  wait_queue_head_t wait;
32  #ifdef WAITQUEUE_DEBUG
33  long __magic;
34  #endif
35  };
-----

```

这两种体系结构对信号量的实现都提供了一个指向 `wait_queue` 的指针和一个计数。计数指的是同一时刻持有信号量的进程的数目。有了信号量,就能够让多个进程同时进入代码的临界区。如果计数被初始化成 1,表示只有一个进程能够进入代码的临界区;计数为 1 的信号量称为互斥信号量 (mutex)。

信号量用 `sema_init()` 来初始化,并分别通过调用 `down()` 和 `up()` 来上锁和解锁。如果进程对一个已上锁的信号量调用 `down()`,它就会阻塞并忽略发送给它的所有信号。当然也存在函数 `down_interruptible()`,如果获得了信号量,`down_interruptible()` 就返回 0;如果进程在阻塞期间被中断,它就返回 `-EINTR`。

当进程调用 `down()` 或 `down_interruptible()` 时,信号量的 `count` 字段递减。如果 `count` 字段小于 0,那么调用 `down()` 的进程被阻塞并被添加到信号量的 `wait_queue`;如果 `count` 字段大于或等于 0,该进程将继续执行。

执行完临界区的代码之后,进程应该调用 `up()` 告知信号量自己已离开临界区。进程调用 `up()`,增加信号量的 `count` 字段,如果 `count` 大于或等于 0,则唤醒在信号量的 `wait_queue` 上等待的某个进程。

7.4 系统时钟：关于时间和定时器

为了进行调度,内核用系统时钟来获知进程运行了多长时间。第 5 章已经将系统时钟作为讨论中断的一个例子进行了介绍。此处将分析实时时钟和它的使用及其实现;首先简单概述一下时钟。

时钟是作用在处理器上的周期性信号,它使得处理器在时域中运行。处理器依靠时钟信号来获知什么时候能够执行它的下一个任务,例如将两个整数相加或从内存取数据。这种时钟信号的速率 (1.4GHz、2GHz, 等等) 曾被用于比较系统在局部电子存储方面的处理速度。

在任何给定的时刻,系统中总有几个时钟或定时器在运行。一些简单的例子包括显示在屏幕右下角的时刻 (也叫做挂钟时间)、光标耐心地在杂乱的桌面上跳动,或者笔记本电脑的屏保由于屏幕的休止而接管了屏幕。更复杂的计时例子包括音频和视频的重放、键的重复 (保持一个键的按下状态)、通信端口的运行速度有多快,以及如前面所讨论的,进程运行了多长时间。

7.4.1 实时时钟：现在几点了

挂钟时间 (wall clock time) 的 Linux 接口通过 `/dev/rtc` 设备驱动程序的 `ioctl()` 函数来实现。使用这个驱动程序的设备称为 RTC (实时时钟)。RTC^① 给计时函数提供一个 114 字节大小的

① RTC 由多个厂家生产,其中最有名的是摩托罗拉的 `mc146818` (这种 RTC 已经不再生产,而是用 Dallas `DS12885` 或同等产品来替代)。

小型用户 NVRAM。该设备的输入是一个 32.768KHz 的振荡器和一个与备用电池的连接。个别的 RTC 模型将振荡器和电池嵌入其中，而其他的 RTC 目前被嵌入处理器芯片集的外围总线控制器（例如，南桥）。RTC 不仅能报告时刻，而且也是一个能中断系统的可编程定时器，其中断频率为 2~8192Hz。RTC 也可以像闹钟那样每日都中断。下面分析 RTC 的代码：

```
-----
#include/linux rtc.h

/*
 * ioctl calls that are permitted to the /dev/rtc interface, if
 * any of the RTC drivers are enabled.
 */

70 #define RTC_AIE_ON      _IO('p', 0x01)    /* Alarm int. enable on */
71 #define RTC_AIE_OFF     _IO('p', 0x02)    /* ... off */
72 #define RTC_UIE_ON      _IO('p', 0x03)    /* Update int. enable on */
73 #define RTC_UIE_OFF     _IO('p', 0x04)    /* ... off */
74 #define RTC_PIE_ON      _IO('p', 0x05)    /* Periodic int. enable on */
75 #define RTC_PIE_OFF     _IO('p', 0x06)    /* ... off */
76 #define RTC_WIE_ON      _IO('p', 0x0f)    /* Watchdog int. enable on */
77 #define RTC_WIE_OFF     _IO('p', 0x10)    /* ... off */

78 #define RTC_ALM_SET      _IOW('p', 0x07, struct rtc_time) /* Set alarm time */
79 #define RTC_ALM_READ     _IOR('p', 0x08, struct rtc_time) /* Read alarm time */
80 #define RTC_RD_TIME      _IOR('p', 0x09, struct rtc_time) /* Read RTC time */
81 #define RTC_SET_TIME     _IOW('p', 0x0a, struct rtc_time) /* Set RTC time */
82 #define RTC_IRQP_READ    _IOR('p', 0x0b, unsigned long) /* Read IRQ rate */
83 #define RTC_IRQP_SET     _IOW('p', 0x0c, unsigned long) /* Set IRQ rate */
84 #define RTC_EPOCH_READ   _IOR('p', 0x0d, unsigned long) /* Read epoch */
85 #define RTC_EPOCH_SET    _IOW('p', 0x0e, unsigned long) /* Set epoch */
86
87 #define RTC_WKALM_SET     _IOW('p', 0x0f, struct rtc_wkalrm) /* Set wakealarm */
88 #define RTC_WKALM_RD     _IOR('p', 0x10, struct rtc_wkalrm) /* Get wakealarm */
89
90 #define RTC_PLL_GET       _IOR('p', 0x11, struct rtc_pll_info) /* Get PLL correction */
91 #define RTC_PLL_SET       _IOW('p', 0x12, struct rtc_pll_info) /* Set PLL correction */
-----
```

ioctl() 控制函数位于文件 include/linux/rtc.h 中。在本书中，对于 PPC 的体系结构来说，并不是所有对 RTC 的 ioctl() 调用都要被实现。这些控制函数都调用低层的、硬件特有的函数（如果已经实现了的话）。本节的例子中使用 RTC_RD_TIME 函数。

下面是利用 ioctl() 调用获得日期和时间的例子。这个程序只是打开驱动程序并向 RTC 硬件查询当前的日期和时间，然后将信息输出到标准错误输出终端 stderr。注意，每次只有一个用户能够访问 RTC 的驱动程序。在讨论驱动程序的代码中强调了这一点。

```
-----
Documentation/rtc.txt
/*
 * Trimmed down version of code in /Documentation/rtc.txt
 *
 */
-----
```



```

int main(void) {

int fd, retval = 0;
//unsigned long tmp, data;
struct rtc_time rtc_tm;

fd = open ("/dev/rtc", O_RDONLY);

/* Read the RTC time/date */
retval = ioctl(fd, RTC_RD_TIME, &rtc_tm);

/* print out the time from the rtc_tm variable */

close(fd);
return 0;

} /* end main */
-----

```

这段代码是/Documentation/rtc.txt 中一个完整例子的片断。在这个程序中，两行主要的代码是 open() 命令和 ioctl() 调用。open() 告诉我们要使用哪一个驱动程序 (/dev/rtc)，ioctl() 则指出一个明确的路径，该路径可到达由 RTC_RD_TIME 命令指明的 RTC 物理接口。open() 命令的驱动程序代码驻留在驱动程序源码中，在这里，讨论它的唯一意义只在于表明打开的是哪一个设备驱动程序。

7.4.2 读取 PPC 的实时时钟

内核编译时，将在内核中插入适当的代码树 (x86、PPC、MIPS，等等)。此处，对于非 x86 系统的通用 RTC 驱动程序，在源代码文件中只讨论 PPC 源码这一分支：

```

-----
/drivers/char/genrtc.c
276 static int gen_rtc_ioctl(struct inode *inode, struct file *file,
277     unsigned int cmd, unsigned long arg)
278 {
279     struct rtc_time wtime;
280     struct rtc_pll_info pll;
281
282     switch (cmd) {
283
284     case RTC_PLL_GET:
285     ...
290     case RTC_PLL_SET:
291     ...
298     case RTC_UIE_OFF: /* disable ints from RTC updates. */
299     ...
302     case RTC_UIE_ON: /* enable ints for RTC updates. */
303     ...

```



```

305 case RTC_RD_TIME: /* Read the time/date from RTC */
306
307     memset(&wtime, 0, sizeof(wtime));
308     get_rtc_time(&wtime);
309
310     return copy_to_user((void *)arg, &wtime, sizeof(wtime)) ? -EFAULT:0;
311
312 case RTC_SET_TIME: /* Set the RTC */
313     return -EINVAL;
314 }
...
353 static int gen_rtc_open(struct inode *inode, struct file *file)
354 {
355     if (gen_rtc_status & RTC_IS_OPEN)
356         return -EBUSY;
357     gen_rtc_status |= RTC_IS_OPEN;
-----

```

这段代码是针对 `ioctl` 命令集的 `case` 语句。因为以 `RTC_RD_TIME` 标志从用户空间的测试程序调用了 `ioctl`，所以控制权被传递到第 305 行。下一个调用的是第 308 行的 `get_rtc_time(&wtime)`，它位于 `rtc.h` 中（见下列代码）。在离开这个代码段前，请留意一下第 353 行。将状态设置为 `RTC_IS_OPEN`，每次只允许一个用户可以通过 `open()` 来访问驱动程序。

```

-----
include/asm-ppc/rtc.h
045 static inline unsigned int get_rtc_time(struct rtc_time *time)
046 {
047     if (ppc_md.get_rtc_time) {
048         unsigned long nowtime;
049
050         nowtime = (ppc_md.get_rtc_time)();
051
052         to_tm(nowtime, time);
053
054         time->tm_year -= 1900;
055         time->tm_mon -= 1; /* Make sure userland has a 0-based month */
056     }
057     return RTC_24H;
058 }
-----

```

内联函数 `get_rtc_time()` 通过第 50 行上的 `ppc_md.get_rtc_time` 调用结构体变量所指向的函数。早在内核初始化时，就已经在 `chrp_setup.c` 中设置了该变量。

```

-----
arch/ppc/platforms/chrp_setup.c
447 chrp_init(unsigned long r3, unsigned long r4, unsigned long r5,
448 unsigned long r6, unsigned long r7)
449 {
...
477     ppc_md.time_init = chrp_time_init;
478     ppc_md.set_rtc_time = chrp_set_rtc_time;

```

```

479  ppc_md.get_rtc_time = chrp_get_rtc_time;
480  ppc_md.calibrate_decr = chrp_calibrate_decr;
-----

```

在下列代码段中，函数 `chrp_get_rtc_time()`（第 479 行）是在 `chrp_time.c` 中被定义的。因为 CMOS 存储器中的时间信息被周期性地更新，所以，这段读代码被封装在 `for` 循环中，如果正在进行更新，则重读该代码块。

```

-----
arch/ppc/platforms/chrp_time.c
122  unsigned long __chrp chrp_get_rtc_time(void)
123  {
124      unsigned int year, mon, day, hour, min, sec;
125      int uip, i;
126      ...
141      for ( i = 0; i<1000000; i++) {
142          uip = chrp_cmos_clock_read(RTC_FREQ_SELECT);
143          sec = chrp_cmos_clock_read(RTC_SECONDS);
144          min = chrp_cmos_clock_read(RTC_MINUTES);
145          hour = chrp_cmos_clock_read(RTC_HOURS);
146          day = chrp_cmos_clock_read(RTC_DAY_OF_MONTH);
147          mon = chrp_cmos_clock_read(RTC_MONTH);
148          year = chrp_cmos_clock_read(RTC_YEAR);
149          uip |= chrp_cmos_clock_read(RTC_FREQ_SELECT);
150          if ((uip & RTC_UIP)==0) break;
151      }
152      if (!(chrp_cmos_clock_read(RTC_CONTROL)
153          & RTC_DM_BINARY) || RTC_ALWAYS_BCD)
154      {
155          BCD_TO_BIN(sec);
156          BCD_TO_BIN(min);
157          BCD_TO_BIN(hour);
158          BCD_TO_BIN(day);
159          BCD_TO_BIN(mon);
160          BCD_TO_BIN(year);
161      }
162      ...
054  int __chrp chrp_cmos_clock_read(int addr)
055  {  if (nvram_as1 != 0)
056      outb(addr>>8, nvram_as1);
057      outb(addr, nvram_as0);
058      return (inb(nvram_data));
059  }
-----

```

最后，在 `chrp_get_rtc_time()` 中，使用函数 `chrp_cmos_clock_read`，从 RTC 设备读取时间结构各成员的值，然后用 `rtc_tm` 结构格式化并返回这些值，`rtc_tm` 结构被传回用户空间测试程序的 `ioctl` 回调。

7.4.3 读取 x86 的实时时钟

在 x86 系统中，读取 RTC 的方法类似于 PPC 中的方法，但是更简洁、更健壮一些。继续分

析打开的驱动程序/dev/rtc，但是这次，编译程序已经为 x86 体系结构编译了文件 rtc.c。此处讨论 x86 源码分支。

```
-----
drivers/char/rtc.c
...
352 static int rtc_do_ioctl(unsigned int cmd, unsigned long arg, int kernel)
353 {
...
switch (cmd) {
...
482 case RTC_RD_TIME: /* Read the time/date from RTC */
483 {
484     rtc_get_rtc_time(&wtime);
485     break;
486 }
...
1208 void rtc_get_rtc_time(struct rtc_time *rtc_tm)
1209 {
...
1238     spin_lock_irq(&rtc_lock);
1239     rtc_tm->tm_sec = CMOS_READ(RTC_SECONDS);
1240     rtc_tm->tm_min = CMOS_READ(RTC_MINUTES);
1241     rtc_tm->tm_hour = CMOS_READ(RTC_HOURS);
1242     rtc_tm->tm_mday = CMOS_READ(RTC_DAY_OF_MONTH);
1243     rtc_tm->tm_mon = CMOS_READ(RTC_MONTH);
1244     rtc_tm->tm_year = CMOS_READ(RTC_YEAR);
1245     ctrl = CMOS_READ(RTC_CONTROL);
...
1249     spin_unlock_irq(&rtc_lock);
1250
1251     if (!(ctrl & RTC_DM_BINARY) || RTC_ALWAYS_BCD)
1252     {
1253         BCD_TO_BIN(rtc_tm->tm_sec);
1254         BCD_TO_BIN(rtc_tm->tm_min);
1255         BCD_TO_BIN(rtc_tm->tm_hour);
1256         BCD_TO_BIN(rtc_tm->tm_mday);
1257         BCD_TO_BIN(rtc_tm->tm_mon);
1258         BCD_TO_BIN(rtc_tm->tm_year);
1259     }
-----
```

测试程序在对驱动程序 rtc.c 的调用中使用了 ioctl() 的 RTC_RD_TIME 标志。接着，ioctl 的 switch 语句用 RTC 的 CMOS 存储器中的内容来填充时间结构。此处是如何读取 RTC 硬件的 x86 实现：

```
-----
include/asm-i386/mc146818rtc.h
...
018 #define CMOS_READ(addr) ({ \
019     outb_p((addr), RTC_PORT(0)); \
```

```
020    inb_p(RTC_PORT(1)); \
021  })
```

7.5 小结

本章介绍了 Linux 的调度程序、内核抢占，以及系统时钟和定时器。

更具体地说，介绍了下列内容。

- 简单介绍了 Linux 2.6 的新调度程序及其新特性。
- 描述了调度程序如何从所有可供选择的进程中选择下一个进程，以及调度程序选择进程时所使用的算法。
- 讨论了调度程序用来真实地交换进程的上下文切换，追踪了从切换函数一直到低层的体系结构特有代码的全过程。
- 介绍了 Linux 中的进程如何通过调用 `schedule()` 为其他进程让出 CPU，以及接下来内核如何把该进程标记为“将要被调度”的进程。
- 深入研究了 Linux 内核如何基于某个进程先前的行为来计算其动态优先级，以及最终如何从调度队列删除进程。
- 分别介绍了隐式和显式的用户级和内核级的抢占，以及在 Linux 2.6 内核中如何处理每一种抢占。
- 分析了定时器和系统时钟，以及在 x86 和 PPC 这两种体系结构中系统时钟分别是如何实现的。

7.6 习题

1. Linux 如何通知调度程序周期性地运行？
2. 请描述交互式进程和非交互式进程之间的区别。
3. 实时进程对于调度程序有何特殊之处？
4. 进程用完调度程序的节拍时会发生什么？
5. O(1)调度程序的优势是什么？
6. 调度程序使用何种数据结构来管理在系统中运行的进程的优先级？
7. 如果在持有自旋锁的情况下调用 `schedule()`，会发生什么？
8. 内核如何确定一个内核进程是否可以被隐式地抢占？

第 8 章

内核引导

本章内容

- BIOS 和 Open Firmware
- 引导加载程序
- 与体系结构相关的内存初始化
- 原始的 RAM 盘
- 开始: `start_kernel()`
- `init` 线程 (或进程 1)

迄今为止, 我们已经探讨过 Linux 内核的一些子系统及以操作为中心的数据结构。每章都假定该子系统已经启动并正在运行, 因而主要关注典型的内核子系统的管理及其处理操作。每个子系统在使用前都必须初始化, 这一初始化过程在内核引导时进行, 引导加载程序将内核映像加载到内存并将控制权交给内核后, 才开始进行初始化。

我们根据实际执行的线性次序跟踪内核的初始化过程。首先讨论加电时发生什么, 一直到调用第一个与体系结构无关的函数 `start_kernel()`, 然后关注该函数的运行情况, 直到调用 `/sbin/init` 为止。图 8-1 说明了从系统加电到断电这一过程中所有事件发生的顺序。

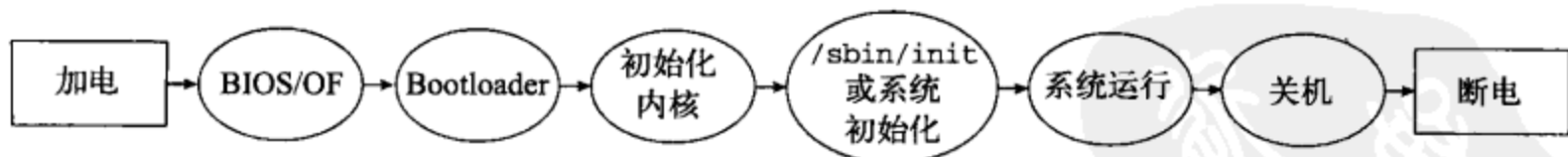


图 8-1 内核的初始状态及其引导过程

首先将讨论 BIOS 和 Open Firmware, 它们分别是 x86 和 PPC 系统加电后最先运行的代码。然后讨论 Linux 系统中常见的引导加载程序, 以及该程序如何加载内核并将执行控制权交给内核。接下来详细讨论内核的初始化过程, 此时要初始化所有子系统。内核初始化的最后一步是由进程 1 调用 `/sbin/init`。某些进程必须在用户登录前运行, `init` 通过将这些进程投入运行来继续系统的初始化。

显然, 内核的部分初始化特性由交叉运行的子系统组成, 因而, 由始至终跟踪给定子系统的整个初始化过程而不被打断, 会非常困难。无论如何, 顺着 Linux 内核引导的线性次序一步一步走下去, 将有利于跟踪内核子系统的实际建立过程, 同时也说明了这个“白手起家”进程的复杂性。

本章将提及前面章节介绍的诸多结构体, 这是因为在本章它们是首次出现并初始化的结构

体。现在，首先要考虑的就是 BIOS 和 Open Firmware。

8.1 BIOS 和 Open Firmware

加电后，处理器首先访问通常位于只读内存中的某一地址。这个只读内存一般是指 Flash ROM（或仅是 Flash），系统上最先运行的代码就存储在这里，这些代码负责启用系统中相应的部分，以便加载内核。

对于 x86 系统而言，这就是系统 BIOS 驻留之处。BIOS（Basic Input Output System，基本输入输出系统）是一段用来引导系统的、与硬件相关的系统初始化代码。在 x86 系统中，引导加载程序和 Linux 依靠 BIOS 将系统带入一个已知的状态。BIOS 的接口是一组统一的名为中断（interrupt）的函数集。内核载入时，Linux 使用这些中断来查询系统的可用资源。BIOS 完成初始化操作后，再从引导设备（参见 8.2 节）复制最初的 512 字节数据到地址 0x7c00 处，并跳转到该地址。虽然在某些安装程序中，BIOS 也通过网络连接来加载操作系统，但从硬盘驱动器加载 Linux 时并不考虑这种情况。加载 Linux 后，BIOS 仍驻留在内存中，其函数也可以被访问，并且能借助中断来调用这些函数。

对 PowerPC 而言，初始化代码的类型与特定 PowerPC 体系结构的出现时间有关。旧式的 IBM 系统使用 PerP（PowerPC Reference Platform，PowerPC 参考平台），当前，更多 IBM 系统使用 CHRP（Common Hardware Reference Platform，通用硬件参考平台）。G4 系统及后续系统“真正新世界”（True New World）以及 OF（Open Firmware）都采用特殊的体系结构实现（关于该处理器和与系统无关的引导固件及其如何使用这些格式，可参见 Open Firmware 的主页 www.openfirmware.org）。

8.2 引导加载程序

引导加载程序（BootLoader）是驻留在计算机引导设备中的程序。第一个引导设备往往是系统中第一个硬盘。完成足够的系统初始化工作（以便支持内存、中断以及加载内核所需的 I/O）后，BIOS（x86 中）或固件（PPC 中）就会调用引导加载程序。成功地将它加载进来以后，内核就开始初始化并配置操作系统。

对 x86 系统而言，BIOS 允许用户为其系统设置引导设备的顺序。典型的引导设备有软盘、光盘和硬盘。格式化磁盘时（例如，使用 fdisk 命令）会创建 MBR（Master Boot Record，主引导记录），该记录存储在引导设备的第一个扇区（0 扇区，0 磁道，0 磁头）。MBR 包含一个小程序和一张四入口点的分区表。引导扇区的末尾在 510 单元处有一个十六进制的标记 0xAA55。表 8-1 给出了 MBR 的组成。

表8-1 MBR的组成

偏移量	长度	作用
0x00	0x1bd	MBR的程序代码
0x1be	0x40	分区表
0x1fe	0x2	十六进制标记或签名

MBR 的分区表保存每个硬盘基本分区的信息。表 8-2 给出了 MBR 分区表的 16 字节表目。

表8-2 MBR的16字节表目

偏移量	长 度	作 用
0x00	1	活动引导分区标志
0x01	3	启动引导分区的柱面/磁头/扇区
0x04	1	分区类型 (Linux使用0x83, 而PPC的PReP使用0x41)
0x05	3	终止引导分区的柱面/磁头/扇区
0x08	4	分区的起始扇区号
0x0c	4	分区长度 (以扇区为单位)

在自检和硬件识别的最后阶段, 系统初始化代码 (固件或 BIOS) 将访问硬件驱动控制器, 以读取。识别出引导驱动器的类型后, 就可以遵循公布的接口 (如 IDE 驱动器) 来访问 0 磁头、0 柱面和 0 扇区。

引导设备定位后, 就将 MBR 复制到内存地址 0x7c00 处并执行。主引导记录开头的一小段程序将自己迁移到别的位置, 并搜索分区表来定位活动引导分区。然后, MBR 将活动引导分区的代码复制到地址 0x7c00 处并开始执行这些代码。由此看来, x86 系统通常引导 DOS, 但是, 活动引导分区可以有一个引导加载程序来依次加载操作系统。下面将讨论 Linux 中最常用的几种引导加载程序。图 8-2 给出了引导时的内存状态。

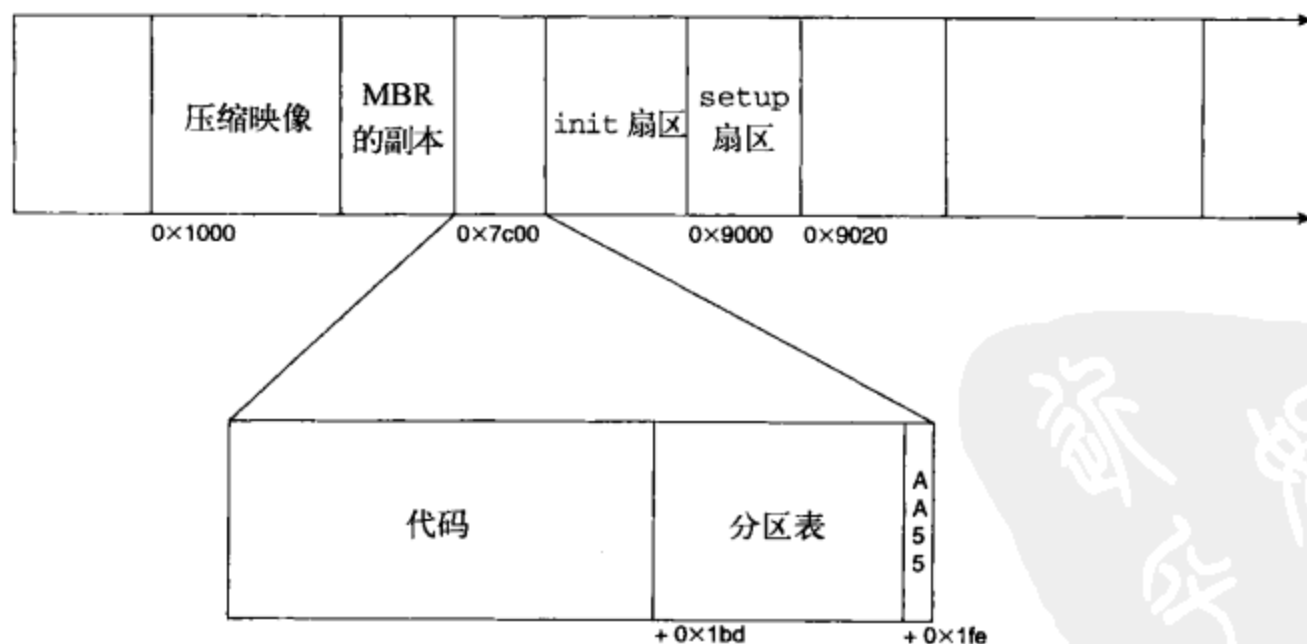


图 8-2 引导时的内存视图

8.2.1 GRUB

GRUB (Grand Unified Bootloader) 是基于 x86 的引导加载程序, 用来加载 Linux。GRUB 2 在设计之初就考虑了移植到 PPC 系统的问题。www.gnu.org/software/grub 上有丰富的相关文档, 包括其发展历史和将来的可扩充性。GRUB 将识别引导驱动器的文件系统, 并且可以通过指定文

件名、驱动器和内核所在的分区来加载内核。GRUB 是一个两阶段引导加载程序^①。BIOS 将调用存储在 MBR 中的第一阶段代码，并由第一阶段将第二阶段的一部分代码装入内存，之后，第二阶段就能借助文件系统来完成自身的加载。GRUB 运行时每一阶段的详细步骤如下。

第一阶段：

- (1) 初始化；
- (2) 检测正在加载的驱动器；
- (3) 加载第一阶段的一个扇区；
- (4) 跳转到第二阶段。

第二阶段：

- (1) 加载第二阶段的剩余代码；
- (2) 跳转到已加载的代码。

通过交互式的命令程序或菜单驱动界面均可访问 GRUB，但使用菜单界面时必须创建一个配置文件。下面是来自加载 Linux 内核的 GRUB 配置文件中的一段代码：

```
-----
/boot/menu.lst
...
title      Kernel 2.6.7, test kernel
root       (hd0,0)
kernel     /boot/bzImage-2.6.7-mytestkernel root=/dev/hda1 ro②
...
-----
```

有如下选项：title，用于保存设置标签；root，用于将当前根设备设置为 hd0 的分区 0；kernel，用于从特定文件中加载主要的内核引导映像。内核入口的其他信息将作为引导参数传给内核。

内核引导的某些方面，例如内核映像加载与解压的位置，将在 Linux 内核代码体与系统结构相关的部分配置。x86 系统中相关内容请查阅 arch/i386/boot/setup.S：

```
-----
arch/i386/boot/setup.S
61 INITSEG = DEF_INITSEG    # 0x9000, we move boot here, out of the way
62 SYSSEG = DEF_SYSSEG      # 0x1000, system loaded at 0x10000 (65536).
63 SETUPSEG = DEF_SETUPSEG  # 0x9020, this is the current segment
-----
```

该配置指定 Linux 将可执行映像加载到线性地址 0x9000 处之后跳转到 0x9020 处。此处，Linux 内核的未压缩部分将压缩部分的代码解压到地址 0x10000 处，并开始初始化内核。

GRUB 基于多重引导规范。本书编写时，Linux 的所有结构并不都是多引导兼容的，但讨论多引导需求依然很有意义。

多重引导规范

多重引导规范 (Multiboot Specification) 描述了任意可能的引导加载程序和任意可能的操作

① 有时，GRUB 也分为 1.5 阶段，但我们仅讨论常用的两级。

② 内核在引导时可以通过内核命令行来接受规范，这是一个描述参数列表的字符串，这些参数用于说明诸如硬件规格、默认值之类的信息。有关 Linux 引导提示的更多信息，可参见 www.tldp.org/HOWTO/BootPrompt-HOWTO.html。